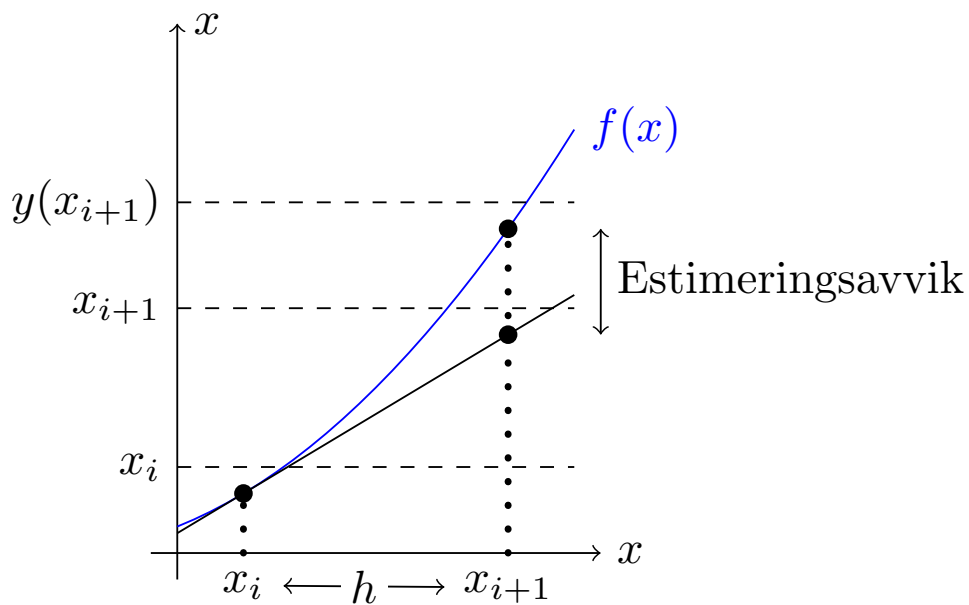


Programmering med Python

Grunnleggende elementer i Python

Tekstprogrammering med Python for MAT202



Amir M. Hashemi

Forord

Dette veiledningsheftet er en del av supplementet til læreboken Matematikk for ingeniørutdanning (1. utgave) [1], og er tilpasset for programmeringsdelen til MAT202.

Anbefaling til leseren

Når du skal lære et nytt emne er det ikke nok å pugge eller lære seg å få tak i hvordan ting skal gjøres. Det er like viktig å spørre seg hvorfor metoden er som den er, og forstå hvordan ting henger sammen. Da blir det lettere å lære. Jo bedre du forstår hvert delemne, desto lettere er det å bruke den til å løse aktuelle problemer. Du får da god erfaring til å anvende den i andre fagfelt.

Takk

En stor takk rettes også til Jonas Gahr Sturtzel Lunde og Kristine Baluka ved UiO, og Jarle Peder Berntsen, Anita Gjesteland ved UiB og Erik Andreas Hanson ved HVL, som har gitt oss tilgang til deres notater, modellerings og programmeringsoppgaver i oppgavesamlingene deres ved UiO og UiB. Jeg vil spesielt sende en stor takk til Erik Andreas Hanson, Marthina Sætre Vahedian, Lorentz T. Syslak, og alle ENT3R-mentorene i Bergen for korrekturlesing av teksten og innspill. Sist og ikke minst vil vi gjerne takke mine kollegaer ved HVL, for å lese gjennom dette heftet, og ha hjulpet å forbedre både teksten og innholdet.

Bergen, 8. mars 2026
Amir M. Hashemi

Eventuelle kommentarer eller meldinger om feil i notatene tas imot med glede: ahas@hvl.no.

© Forfatteren 2022–2030.

Innhold

1	Programmering i MAT202	3
1.1	Funksjoner av flere variable	3
1.2	Differensialligninger	8
1.3	Eulers metode	8
1.4	Eulers metode for systemer (ikke vektlagt)	16
1.5	Oppgaver og programmeringsoppgaver i kapittel 9	19
2	Litt programmering med Python	23
2.1	Å komme i gang med Python	23
2.2	Noen viktige elementer i Python-programmering	24
2.3	Noen av Pythons innebygde funksjoner	25
2.4	Grunnleggende Python-programmering	26
2.5	Datatyper	30
2.6	Variabler	31
2.7	Å sende inn datasett i Python-program	32
2.8	Importering av pakker (numpy, math, pylab, sympy)	34
2.9	Indeksring	36
2.10	Løkker (betingelser og gjentakelser)	37
2.11	Funksjoner	42
2.12	Plotting i Python	49
2.13	Lister	53
2.14	Python-filhåndtering	55
2.15	Laste inn, håndtere og plotte datasett	56
2.16	Modellering: tekopp-temperatur	56
2.17	Oppsummering	59
	Fasit	62

Programmering i matematikk 2 (MAT202)

Vi leser i emnebeskrivelsen til matematikk 2 (MAT202) at *etter å ha fullført dette emnet kan kandidaten identifisere og løse problem relatert til delemne i innhold og ha kjennskap til numerisk løsning av problem ved hjelp av programmering.*

Det forventes dermed ikke at studentene skal kunne programmere fra bunnen av (se oppgave 1.3.1 og 1.5.4). Derimot skal de ha *kjennskap* til numerisk problemløsning ved hjelp av programmering. Med kjennskap menes det at studentene *kan forklare hva programmet utfører generelt, og detaljene i programmet, kan forklare hvilket problem programmet løser, kan forklare hva kan endres om mulig i programmet for å øke presisjonen i programmet, og kan tilpasse programmet til en lignende problemstilling (modifisere).*

Derfor legges ikke noe i formelarket som tildeles med eksamensoppgavene noe med programmering.

Vi skal bruke Python i matematikk 2, og i følgende temaer kan programmering være aktuelt:

- å skissere en flate på formen $z = f(x, y)$
- å løse differensialligninger ved å bruke Eulers metode, en numerisk metode til å regne en tilnærmet løsning til:

$$\frac{dy}{dx} = f(x, y), \text{ gitt } y(x_0) = y_0$$

Bestem $y_i = y(x_i)$, det vil si funksjonsverdi y_i for en gitt x -verdi x_i .

- man kan eventuelt lese et gitt program og forklare hva programmet utfører generelt og i detalj, hvilket problem programmet løser, hva kan endres om mulig i programmet for å øke presisjonen i programmet, og kan tilpasse programmet til en lignende problemstilling.

Kort sagt: Det forventes ikke at studentene lager et program fra scratch, men viser forståelse for hvordan koden henger sammen, og hvordan kan forbedres i presisjon, og tilpasses til en lignende problemstilling.

Oversikt

Her er oversikt over innhold i dette heftet:

Kapittel 1: Programmering i matematikk 2

Kapittel 2: Grunnleggende elementer i Python programmering

Så hvis du vil få litt mer ut fra programmeringsdelen i matematikk 2, kan du gjerne lese kapittel 2 også. Før du leser kapittel 2, kan du lese neste side om PRIMM-metoden.

PRIMM-metoden for å tilrettelegge tekstprogrammering

Her har vi benyttet PRIMM-metoden for å tilrettelegge for læring av tekstprogrammering. PRIMM står for Predict-Run-Investigate-Modify-Make. PRIMM ble introdusert av Sue Sentance i 2017 og trekker sammen kjente effektive undervisningspraksiser i ett strukturert rammeverk.

- **Predict-fasen:** Forklar hva programmet utfører og hvilket problem programmet prøver å løse.
- **Run-fasen:** Kjør programmet for å lære hva programmet utfører, og eventuelt debugge (finne feilen) i programmet.
- **Inverstigat-fasen:** som bør varieres – gode øvelser som fremmer detaljene i programmering, og man lærer om syntaksene og kommandoene steg for steg. Oppgaver der det er lagt inn feil i programmet og man skal finne feilen (feilsøking/debugging) i programmet.
- **Modify-fasen:** Man skal lære hva programmet utfører, og lære seg å gjøre nødvendige endringer for å tilpasse programmet til å løse et nytt problem. Man kan ha oppgaver, der man skal endre litt, og kan ha oppgaver der man skal endre flere deler for å få tilpasset til en ny problemstilling.
- **Make-fasen:** Når man har gått gjennom de første fire trinnene, kan det gis oppgaver der man kan låne kodebiter fra det originale programmet, men det vil ha en ny funksjon, kontekst eller problem som skal løses. Disse fem fasene er kanskje ikke i hver leksjon, avhengig av emnet. Og du kan sykle gjennom de tre, fire første fasene eller alle fem fasene flere ganger.

Å komme i gang med Python

Det er to hovedversjoner av Python, Python 2 og Python 3. Disse versjonene er ganske forskjellige. Her bruker vi Python 3, fordi den er mer semantisk korrekt og støtter nyere funksjoner.

Python er et tekstbasert programmeringsspråk, og for å komme igang med å programmere, så har man flere alternativer blant annet:

- Programmere på nett: Det er flere tjenester på nettet som lar deg skrive og kjøre python-programmer, som for eksempel:

<https://trinket.io/python3>

<https://replit.com/new/python3>

- Installere en kildekode-plattform som både er en kode-editor, men som har innebygget en egen «Python-motor».

<https://www.spyder-ide.org/>

<https://jupyter.org/>

Eksempel på et slikt program er Spyder. Dette er en åpen kildekode-plattform integrert i utviklingsmiljø for vitenskapelig programmering med Python. Spyder integreres med en rekke fremtredende pakker i den vitenskapelige Python-pakken, inkludert numpy, scipy, Matplotlib, pandas, IPython, sympy og Cython, samt annen programvare med åpen kildekode. Det finnes også andre slike program:

<https://www.anaconda.com/>

<https://codewith.mu/en/download>

Hvis du vil lære først mer om programmering, kan du gjerne lese først kapittel 2.

Det forventes ikke at studentene skriver et program fra scratch, men viser forståelse for hvordan koden henger sammen, og hvordan kan forbedres i presisjon, og tilpasses til en lignende problemstilling.

Kapittel 1

Programmering i MAT202

1.1 Funksjoner av flere variable

Her skal vi se på noen eksempler der man tegner en funksjon med flere variabler.

Her tar vi først et eksempel på hvordan vi kan lage en funksjon som kan brukes flere ganger i programkoden.

For eksempel kan vi skrive en funksjon som kan regne ut volumet av en sylinder med radius r og høyde h .

```
1 import numpy as np
2
3 def Volumsyylinder(r, h):
4
5     volum=np.pi*(r**2)*h
6     return volum
7
8 V=Volumsyylinder(2,3)
9 print(V)
```

Her er et eksempel for en funksjon med flere variabler.

```
1 def f(t,y,z,w):
2     u = 2*y*t -4*np.cos(z) +2
3     return u
```

Plotting av flater $z = f(x, y)$

3D-plott er svært viktige verktøy for å visualisere data som har tre dimensjoner, for eksempel data som har to avhengige og en uavhengig variabel. Ved å plote data i 3d-plott kan vi få en dypere forståelse av data som har tre variabler. Vi kan bruke ulike `matplotlib`-biblioteksfunksjoner til å plote 3D-plott (se også avsnitt 2.12).

- Dele opp ønsket definisjonsområdet med et bestemt antall punkter (n) ved hjelp av `meshgrid`:

```
1 # Laster ned de nødvendige bibliotekene
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 n=101
6 # Setter opp definisjons området til de uavhengige x og y variablene, og
   intervallene er delt i 101 deler
7 X, Y = np.meshgrid(np.linspace(-8, 8, n), np.linspace(-8, 8, n))
```

- Definerer funksjonen

```
1 def f(x, y):
2     t=np.sqrt(x ** 2 + y ** 2)
3     z= t*np.sin(t)
4     return z
```

Aktiverer aksene:

```
1 fig = plt.figure(figsize=(10,8))
2 ax = fig.add_subplot(111, projection = '3d')
```

Plotter overflaten, og setter opp fargelinjen, som er en visualisering av kartleggingen fra skalarverdier til farger.

```
1 surf=ax.plot_surface(X, Y, Z, cmap='jet', alpha=0.8)
2
3 # Setter opp fargelinjen
4 fig.colorbar(surf, shrink=0.4, aspect=7)
```

Setter opp aksene med tekst, og justerer plasseringen:

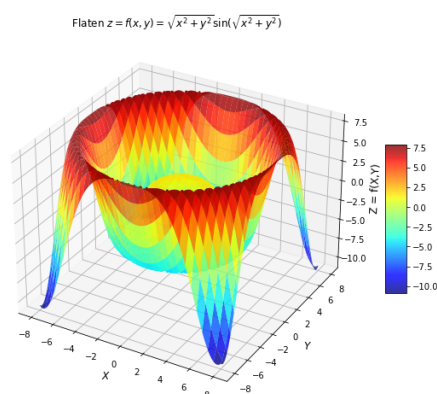
```
1 ax.set_xlabel('$X$', fontsize=12)
2 ax.set_ylabel('$Y$', fontsize=12)
3 ax.set_zlabel('$Z = f(X,Y)$', fontsize=12)
4 # ax.set_title('Flaten $z=f(x,y)$');
5
6 # justerer plasseringene til titel og aksene
7 ax.set_title(" " * 35 + "Flaten $z=f(x,y)$" + " " * 35)
```

Eksempel 1.1.1: Skisser flaten $z = \sqrt{x^2 + y^2} \sin(\sqrt{x^2 + y^2})$, for $-8 \leq x \leq 8$ og $-8 \leq y \leq 8$.

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 n=101
5 # Setter opp definisjons området til de uavhengige x og y variablene:
6 X, Y = np.meshgrid(np.linspace(-8, 8, n), np.linspace(-8, 8, n))
7
8 # Setter opp funksjonen
9
10 def f(x, y):
11     t=np.sqrt(x ** 2 + y ** 2)
12     z= t*np.sin(t)
13     return z
14
15 Z= f(X,Y)
16
17 # Det fungerer også med:
18 # t = np.sqrt(X**2 + Y**2)
19 # Z = t*np.sin(t)
20
21 fig = plt.figure(figsize=(10,8))
22 ax = fig.add_subplot(111, projection = '3d')
23
24 # 3D flate
25 surf=ax.plot_surface(X, Y, Z, cmap='jet', alpha=0.8)
26
27 # colorbar
28 fig.colorbar(surf, shrink=0.4, aspect=7)
29
30 # Setter opp tekst på aksene og grafen
31
32 ax.set_xlabel('$X$', fontsize=12)
33 ax.set_ylabel('$Y$', fontsize=12)
34 ax.set_zlabel('$Z = f(X,Y)$', fontsize=12)
35 # ax.set_title('Flaten $z=f(x,y)$');
36
37 # justerer plasseringene til titel og aksene
38 ax.set_title(" " * 35 + "Flaten $z=f(x,y)$" + " " * 35)
39
40 plt.show()

```

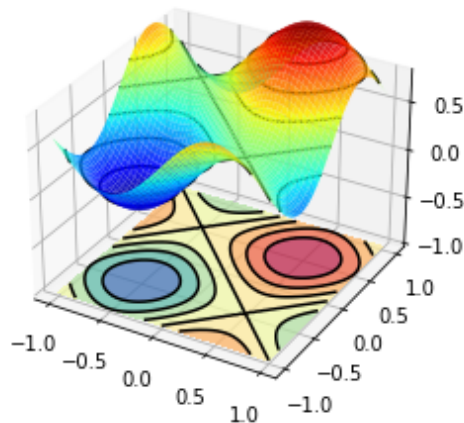


Plotting av konturlinjer (kotekurver) (ikke vektlagt men fint å ha kjenneskap)

Projeksjon av nivå kurver, som er snittskurver mellom flaten $z = f(x, y)$ og et plan loddrett på z -aksen, kalles kotekurver.

3D-konturer refererer til linjene eller kurvene som viser formen og høyden til objekter i et tredimensjonalt rom. Disse konturene hjelper oss å forstå hvor høye eller lave ulike deler av et objekt er. De brukes ofte i felt som geografi, ingeniørfag og kunst for å representere formen til ting på en mer detaljert måte.

For eksempel, hvis du har et fjell, vil 3D-konturene vise bakkene, dalene og toppene fra alle kanter. På samme måte, hvis du har en skulptur av et dyr, vil 3D-konturene beskrive formen på kroppen, hodet og lemmer fra forskjellige synsvinkler.



Nivåkurvene på flaten $z = f(x, y)$ kan illustreres ved:

```
1 ax.contour(X, Y, Z, lw = 2, colors = 'black')
```

Ved å ta `offset = -1` med, kan kotekurvene (konturkurvene på xy -planet) på xy -planet illustreres:

```
1 ax.contour(X, Y, Z, lw = 2, colors = 'black', offset = -1)
2 ax.contourf(X, Y, Z, cmap = 'Spectral_r', offset = -1, alpha = 0.75)
```

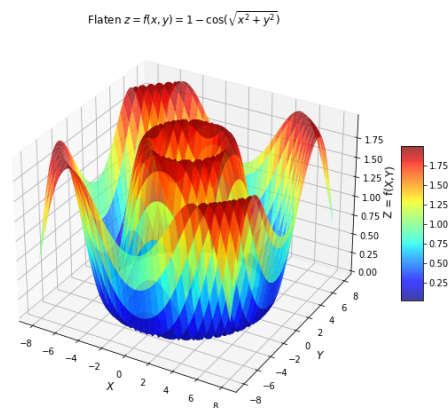
For å gjøre synlig kotekurvene, er det tatt først konturlinjene i svart farge (`colors = 'black'`, og deretter fargelagt dem (`cmap = 'Spectral_r'`).

Eksempel 1.1.2: Her skal vi skissere flaten $z = 0,6 + 0,4(\sin(n\pi x) + \cos(n\pi x))$, og kotekurvene.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Sample data
5 X, Y = np.meshgrid(np.linspace(-1, 1, 50),
6                   np.linspace(-1, 1, 50))
7 def f(x, y):
8     z=np.sin(np.pi*x)+np.sin(np.pi*y)
9     return z
10
11 Z=0.6+0.4*f(X,Y)
12
13 fig = plt.figure()
14 ax = fig.add_subplot(projection = '3d')
15
16 # 3D surface plot
17 ax.plot_surface(X, Y, Z, cmap = 'jet', alpha = 0.9)
18
19 # Filled contour with contour lines
20 ax.contour(X, Y, Z, lw = 2, colors = 'black')
21 ax.contour(X, Y, Z, lw = 2, colors = 'black', offset = -1)
22 ax.contourf(X, Y, Z, cmap = 'Spectral_r', offset = -1, alpha = 0.75)
23
24 # Z-axis limits
25 ax.set(zlim = (-1, 1))
26
27 plt.show()
```

Figuren med flaten, konturkurvene, og kotekurvene er på forrige side. ◇

Oppg. 1.1.1: Skisser flaten beskrevet av $z = f(x, y) = (x^2 + y^2) \cos(x)$, der $-2 \leq y \leq 2$ og $-2 \leq x \leq 2$.
Programmet fi fasit. Flaten ser slik ut:



1.2 Differensialligninger

En differensialligning kan beskrive endringen til en uavhengig variabel (y) i tid (t) og/eller rom (x). Differensialligninger er en dynamisk beskrivelse av et kontinuerlig system eller en prosess, basert på de "likevektsligningene" (balanseligningene) vi har satt opp for prosessen. Differensialligninger kan brukes til å beskrive fenomener i naturen.

En del differensialligninger kan løses ved integrering eller antalytiske metoder, men i mange praktiske anvendelser er differensialligningene ikke analytisk løsbare og kan bare løses numerisk med programmering. Det er sjeldent man kan finne en eksplisitt løsning for en differensialligning. Det finnes flere numeriske metoder for å finne en tilnærmet løsning for en del ordinære differensialligninger. En av dem er Eulers metode som er vektlagt i matematikk 2.

I MAT202 skal vi se nærmere på differensialligninger på formen:

(I) Seperable differensialligninger: $\frac{dy}{dx} = f(x)g(y)$, (EP-1.4)

(II) $y' + P(x)y = Q(x)$ (integrende faktor), der P og Q er kontinuerlige. (EP-1.5)

(III) Eulers metode $\frac{dy}{dx} = f(x, y)$, gitt $y(x_0) = y_0$. Bestem $y_i = y(x_i)$. (EP-2.4)

(IV) $ay' + by = f(x)$ (ved hjelp av to metoder (I) og (V))

(V) $ay'' + by' + cy = f(x)$ (EP-kap. 5.1-5.5)
og deres anvendelser.

I programmeringsdel skal vi fokusere på numerisk løsning av Eulers metode som er vektlagt i MAT202 pensum.

1.3 Eulers metode

Eulers metode, oppkalt etter den sveitsiske matematikeren Leonhard Euler, er en numerisk metode til å regne en tilnærmet løsning til:

$$\frac{dy}{dx} = f(x, y), \text{ gitt } y(x_0) = y_0$$

Bestem $y_i = y(x_i)$.

Differensialligningen kan vise i hvilken retning tangenten til funksjonskurven peker til enhver tid. I Eulers metode kan man tilnærme kurven til løsningen ved tangenten i hvert intervall (det vil si ved en sekvens av korte linjestykker), i steglengde på h . Hvis du bruker mindre steglengder, øker nøyaktigheten av tilnærmingen.

Metoden bygges på tilvekstformelen: $\frac{y(x_n + h) - y(x_n)}{h} \simeq y'(x_n) = f(x_n, y_n)$ som skrives som: $y(x_n + h) \simeq y(x_n) + hf(x_n, y_n)$.

Med andre ord, kan man bevege seg langs tangenten for å bestemme neste y -verdi i hvert steg:

$y_1 = y_0 + hy'_0$: y_1 beregnes og da kan man bestemme $x_1 = x_0 + h$ og $y'_1 = f(x_1, y_1)$ for neste steg:

$y_2 = y_1 + hy'_1$: y_2 beregnes og da kan man bestemme $x_2 = x_1 + h$ og $y'_2 = f(x_2, y_2)$ for neste steg

Denne formelen kan også settes opp ved Taylorsformel: $y(x_n + h) \simeq y(x_n) + hf(x_n, y_n) + O(h^2)$. Eulers metode er en 1. ordens metode da uttrykket for $y_n(x)$ er av orden h (1. orden av h). Metoden har en global feil av orden h og en lokal av orden h^2 .

Eksempel 1.3.1: Bruk Eulers metode med steglengde $h = 0.1$ til å bestemme en tilnærmet verdi for $y(1.3)$ gitt startverdi problemet:

$$\frac{dy}{dt} = 0.2y, \quad y(1) = 2$$

Programmet nedenfor løser dette problemet numerisk:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(t, y):
5     return 0.2*y
6
7 tstart = 1
8 tslutt = 1.3
9
10 h = 0.1
11 N = 3
12
13 t = np.zeros(N+1)
14 for i in range(N+1):
15     t[i] = tstart + i*h
16
17 y = np.zeros(N+1)
18 y[0] = 2
19
20 print("Eulers metode for dy/dt = 0.2y, y(1)=2")
21 print("-" * 50)
22
23 for i in range(N):
24     y[i+1] = y[i] + f(t[i], y[i]) * h
25     print(f"n={i}   t[{i}]=t[{i}]:7.4f}   y[{i}]=y[{i}]:7.4f}   y[{i+1}]=y[{i+1}]:7.4f}")
26
27 print("=" * 50)
28 print(f"\nDen numeriske løsningen gir at y({tslutt}) = {y[N]:.6f}")
29 print(f"Antall steg brukt: {N}")
30 print(f"Steglengde h: {h:.4f}")
31
32 # Eksakt løsning for sammenligning
33 def y_eksakt(t):
34     return 2*np.exp(0.2*(t-1))
35
36 print(f"Eksakt løsning: y({tslutt}) = {y_eksakt(tslutt):.6f}")
37 print(f"Feil: {abs(y_eksakt(tslutt) - y[N]):.6f}")
```

Løsning:

a) Forklar hva programmet utfører

Programmet løser differensialligningen $\frac{dy}{dt} = 0.2y$ med startbetingelsen $y(1) = 2$ på intervallet $[1, 1.3]$ ved hjelp av Eulers metode.

- Funksjonen $f(t, y)$ returnerer den deriverte $0.2y$.
- Startbetingelsen er $t_{\text{start}} = 1$ og $y(1) = 2$, og vi ønsker å nå $t_{\text{slutt}} = 1.3$.
- Med steglengde $h = 0.1$ blir antall steg $N = \frac{1.3-1}{0.1} = 3$.
- Det lages en array t med punktene $t = 1.0, 1.1, 1.2, 1.3$.
- Eulers metode: $y_{n+1} = y_n + h \cdot f(t_n, y_n)$ brukes i en løkke.
- For hvert steg skrives n, t_n, y_n og y_{n+1} ut.

- Til slutt skrives resultatet $y(1.3)$ ut, sammen med antall steg og steglengde, og sammenlignes med eksakt løsning.

b) Regn ut selv med Eulers metode med steglengde $h = 0.1$

Vi har:

$$f(t, y) = 0.2y, \quad h = 0.1, \quad t_0 = 1, \quad y_0 = 2$$

Eulers metode: $y_{n+1} = y_n + h \cdot f(t_n, y_n) = y_n + 0.1 \cdot (0.2y_n) = y_n + 0.02y_n = 1.02y_n$

n	t_n	y_n	$y_{n+1} = 1.02 \cdot y_n$
0	1.0	2.000000	$1.02 \cdot 2.000000 = 2.040000$
1	1.1	2.040000	$1.02 \cdot 2.040000 = 2.080800$
2	1.2	2.080800	$1.02 \cdot 2.080800 = 2.122416$
3	1.3	2.122416	

Den numeriske løsningen med Eulers metode gir $y(1.3) \approx 2.122416$.

c) Regn den eksakte løsningen og sammenlign svarene

Differensialligningen $\frac{dy}{dt} = 0.2y$ er separabel:

$$\begin{aligned} \frac{dy}{dt} &= 0.2y \\ \frac{1}{y} dy &= 0.2 dt \\ \int \frac{1}{y} dy &= \int 0.2 dt \\ \ln |y| &= 0.2t + C \\ y(t) &= Ce^{0.2t} \end{aligned}$$

Bruker startbetingelsen $y(1) = 2$:

$$\begin{aligned} 2 &= Ce^{0.2 \cdot 1} = Ce^{0.2} \\ C &= 2e^{-0.2} \approx 2 \cdot 0.818731 = 1.637462 \end{aligned}$$

Dermed er den eksakte løsningen:

$$y(t) = 2e^{0.2(t-1)}$$

Ved $t = 1.3$:

$$y(1.3) = 2e^{0.2 \cdot 0.3} = 2e^{0.06} \approx 2 \cdot 1.061837 = 2.123674$$

Sammenligning:

Løsning	$y(1.3)$
Eulers metode ($h = 0.1$)	2.122416
Eksakt løsning	2.123674
Feil	0.001258

Eulers metode gir en tilnærming som er svært nær den eksakte løsningen for denne differensialligningen. $\diamond$

Eksempel 1.3.2: Bruk Eulers metode med steglengde $h = 0.5$ til å bestemme en tilnærmet verdi for $y(2)$ gitt startverdi problemet:

$$\frac{dy}{dx} = 0.1 \cdot x \cdot y, \quad y(0) = 1$$

Programmet nedenfor løser dette problemet numerisk:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x, y):
5     return 0.1 * x * y
6
7 xstart = 0
8 xslutt = 2
9
10 h = 0.5
11 N = 4
12
13 x = np.zeros(N+1)
14 for i in range(N+1):
15     x[i] = xstart + i*h
16
17 y = np.zeros(N+1)
18 y[0] = 1
19
20 print("Eulers metode for dy/dx = 0.1*x*y, y(0)=1")
21 print("-" * 50)
22
23 for i in range(N):
24     y[i+1] = y[i] + f(x[i], y[i]) * h
25     print(f"n={i}   x[{i}]=x[i]:7.4f   y[{i}]=y[i]:7.4f   y[{i+1}]=y[i+1]:7.4f")
26
27 print("=" * 50)
28 print(f"\nDen numeriske løsningen gir at y({xslutt}) = {y[N]:.6f}")
29 print(f"Antall steg brukt: {N}")
30 print(f"Steglengde h: {h:.4f}")
31
32 # Eksakt løsning for sammenligning
33 def y_eksakt(x):
34     return np.exp(0.05 * x**2)
35
36 print(f"Eksakt løsning:      y({xslutt}) = {y_eksakt(xslutt):.6f}")
37 print(f"Feil:                    {abs(y_eksakt(xslutt) - y[N]):.6f}")
```

Løsning:

a) Forklar hva programmet utfører

Programmet løser differensialligningen $\frac{dy}{dx} = 0.1xy$ med startbetingelsen $y(0) = 1$ på intervallet $[0, 2]$ ved hjelp av Eulers metode.

- Funksjonen $f(x, y)$ returnerer den deriverte $0.1xy$.
- Startbetingelsen er $x_{\text{start}} = 0$ og $y(0) = 1$, og vi ønsker å nå $x_{\text{slutt}} = 2$.
- Med steglengde $h = 0.5$ blir antall steg $N = \frac{2-0}{0.5} = 4$.
- Det lages en array x med punktene $x = 0.0, 0.5, 1.0, 1.5, 2.0$.
- Eulers metode: $y_{n+1} = y_n + h \cdot f(x_n, y_n)$ brukes i en løkke.
- For hvert steg skrives n , x_n , y_n og y_{n+1} ut.

- Til slutt sammenlignes den numeriske løsningen med den eksakte løsningen $y(x) = e^{0.05x^2}$.

b) Regn ut selv med Eulers metode med steglengde $h = 0.5$

Vi har:

$$f(x, y) = 0.1xy, \quad h = 0.5, \quad x_0 = 0, \quad y_0 = 1$$

Eulers metode: $y_{n+1} = y_n + h \cdot f(x_n, y_n) = y_n + 0.5 \cdot (0.1 \cdot x_n \cdot y_n) = y_n + 0.05 \cdot x_n \cdot y_n = y_n(1 + 0.05x_n)$

n	x_n	y_n	$f(x_n, y_n) = 0.1x_n y_n$	$y_{n+1} = y_n + 0.5 \cdot f(x_n, y_n)$
0	0.0	1.000000	$0.1 \cdot 0.0 \cdot 1 = 0$	$1.000000 + 0.5 \cdot 0 = 1.000000$
1	0.5	1.000000	$0.1 \cdot 0.5 \cdot 1 = 0.05$	$1.000000 + 0.5 \cdot 0.05 = 1.025000$
2	1.0	1.025000	$0.1 \cdot 1.0 \cdot 1.025 = 0.1025$	$1.025000 + 0.5 \cdot 0.1025 = 1.076250$
3	1.5	1.076250	$0.1 \cdot 1.5 \cdot 1.07625 = 0.161438$	$1.076250 + 0.5 \cdot 0.161438 = 1.156969$
4	2.0	1.156969		

Den numeriske løsningen med Eulers metode gir $y(2.0) \approx 1.156969$.

c) Regn den eksakte løsningen og sammenlign svarene

Differensialligningen $\frac{dy}{dx} = 0.1xy$ er separabel:

$$\begin{aligned} \frac{dy}{dx} &= 0.1xy \\ \frac{1}{y} dy &= 0.1x dx \\ \int \frac{1}{y} dy &= \int 0.1x dx \\ \ln |y| &= 0.1 \cdot \frac{x^2}{2} + C = 0.05x^2 + C \\ y(x) &= Ce^{0.05x^2} \end{aligned}$$

Bruker startbetingelsen $y(0) = 1$:

$$\begin{aligned} 1 &= Ce^0 = C \cdot 1 \\ C &= 1 \end{aligned}$$

Dermed er den eksakte løsningen:

$$y(x) = e^{0.05x^2}$$

Ved $x = 2.0$:

$$y(2.0) = e^{0.05 \cdot 4} = e^{0.2} \approx 1.221403$$

Sammenligning:

Løsning	$y(2.0)$
Eulers metode ($h = 0.5$)	1.156969
Eksakt løsning	1.221403
Feil	0.064434

Eulers metode gir en tilnærming som er mindre enn den eksakte løsningen for denne differensialligningen. $\diamond$

Eksempel 1.3.3: Bruk Eulers metode med steglengde $h = 0.1$ til å bestemme en tilnærmet verdi for $y(0.5)$ gitt startverdi problemet:

$$\frac{dy}{dx} = 3x^2y, \quad y(0.1) = 2$$

Programmet nedenfor løser dette problemet numerisk:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x, y):
5     return 3 * x**2 * y
6
7 xstart = 0.1
8 xslutt = 0.5
9
10 h = 0.1
11 N = 4
12
13 x = np.zeros(N+1)
14 for i in range(N+1):
15     x[i] = xstart + i*h
16
17 y = np.zeros(N+1)
18 y[0] = 2
19
20 print("Eulers metode for dy/dx = 3x^2*y, y(0.1)=2")
21 print("-" * 50)
22
23 for i in range(N):
24     y[i+1] = y[i] + f(x[i], y[i]) * h
25     print(f"n={i}   x[{i}]= {x[i]:7.4f} y[{i}]= {y[i]:7.4f} y[{i+1}]= {y[i+1]:7.4f}")
26
27 print("=" * 50)
28 print(f"\nDen numeriske løsningen gir at y({xslutt}) = {y[N]:.6f}")
29 print(f"Antall steg brukt: {N}")
30 print(f"Steglengde h: {h:.4f}")
31
32 # Eksakt løsning for sammenligning
33 def y_eksakt(x):
34     return 2 * np.exp(x**3 - 0.001)
35
36 print(f"Eksakt løsning:      y({xslutt}) = {y_eksakt(xslutt):.6f}")
37 print(f"Feil:                  {abs(y_eksakt(xslutt) - y[N]):.6f}")
```

Løsning:

a) Forklar hva programmet utfører

Programmet løser differensialligningen $\frac{dy}{dx} = 3x^2y$ med startbetingelsen $y(0.1) = 2$ på intervallet $[0.1, 0.5]$ ved hjelp av Eulers metode.

- Funksjonen $f(x, y)$ returnerer den deriverte $3x^2y$.
- Startbetingelsen er $x_{\text{start}} = 0.1$ og $y(0.1) = 2$, og vi ønsker å nå $x_{\text{slutt}} = 0.5$.
- Med steglengde $h = 0.1$ blir antall steg $N = \frac{0.5-0.1}{0.1} = 4$.
- Det lages en array x med punktene $x = 0.1, 0.2, 0.3, 0.4, 0.5$.
- Eulers metode: $y_{n+1} = y_n + h \cdot f(x_n, y_n)$ brukes i en løkke.
- For hvert steg skrives n, x_n, y_n og y_{n+1} ut.

- Til slutt sammenlignes den numeriske løsningen med den eksakte løsningen $y(x) = 2e^{x^3-0.001}$.

b) Regn ut selv med Eulers metode med steglengde $h = 0.1$

Vi har:

$$f(x, y) = 3x^2y, \quad h = 0.1, \quad x_0 = 0.1, \quad y_0 = 2$$

n	x_n	y_n	$f(x_n, y_n) = 3x_n^2y_n$	$y_{n+1} = y_n + h \cdot f(x_n, y_n)$
0	0.1	2.000000	$3 \cdot (0.1)^2 \cdot 2 = 3 \cdot 0.01 \cdot 2 = 0.06$	$2.000000 + 0.1 \cdot 0.06 = 2.006000$
1	0.2	2.006000	$3 \cdot (0.2)^2 \cdot 2.006 = 3 \cdot 0.04 \cdot 2.006 = 0.240720$	$2.006000 + 0.1 \cdot 0.24072 = 2.030072$
2	0.3	2.030072	$3 \cdot (0.3)^2 \cdot 2.030072 = 3 \cdot 0.09 \cdot 2.030072 = 0.548119$	$2.030072 + 0.1 \cdot 0.548119 = 2.084884$
3	0.4	2.084884	$3 \cdot (0.4)^2 \cdot 2.084884 = 3 \cdot 0.16 \cdot 2.084884 = 1.000744$	$2.084884 + 0.1 \cdot 1.000744 = 2.184958$
4	0.5	2.184958		

Den numeriske løsningen med Eulers metode gir $y(0.5) \approx 2.184958$.

c) Regn den eksakte løsningen og sammenlign svarene

Differensialligningen $\frac{dy}{dx} = 3x^2y$ er separabel:

$$\begin{aligned} \frac{dy}{dx} &= 3x^2y \\ \frac{1}{y} dy &= 3x^2 dx \\ \int \frac{1}{y} dy &= \int 3x^2 dx \\ \ln |y| &= x^3 + C \\ y(x) &= Ce^{x^3} \end{aligned}$$

Bruker startbetingelsen $y(0.1) = 2$:

$$\begin{aligned} 2 &= Ce^{(0.1)^3} = Ce^{0.001} \\ C &= 2e^{-0.001} \approx 1.998001 \end{aligned}$$

Dermed er den eksakte løsningen:

$$y(x) = 2e^{x^3-0.001}$$

Ved $x = 0.5$:

$$y(0.5) = 2e^{(0.5)^3-0.001} = 2e^{0.125-0.001} = 2e^{0.124} \approx 2 \cdot 1.132024 = 2.264048$$

Sammenligning:

Løsning	$y(0.5)$
Eulers metode ($h = 0.1$)	2.184958
Eksakt løsning	2.264048
Feil	0.079090

Eulers metode gir en tilnærming som er mindre enn den eksakte løsningen for denne differensialligningen. $\diamond$

Oppg. 1.3.1: Et program er gitt som:

```
1 # Program for Eulers metode med 2 steg
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def f(x, y):
6     return 2 * y
7
8 xstart = 0
9 xslutt = 0.5
10
11 # Med 2 steg
12 h = 0.25
13 N = 2
14
15 x = np.zeros(N + 1)
16 for i in range(N + 1):
17     x[i] = xstart + i * h
18
19 y = np.zeros(N + 1)
20 y[0] = 1
21
22 print("Med 2 steg (h = 0.25):")
23 print('\t x[n] \t y[n] \t y[n+1]')
24
25 for i in range(N):
26     y[i + 1] = y[i] + f(x[i], y[i]) * h
27     print(f"n={i} x[{i}]={x[i]:.4f} y[{i}]={y[i]:.4f} y[{i+1]}={y[i+1]:.4f}")
28
29 print("\n" + "=" * 45)
30 print(f"Numerisk losning: y({xslutt}) = {y[N]:.6f}")
31 print(f"Eksakt losning: y({xslutt}) = {np.exp(2 * xslutt):.6f}")
```

- a) Hvilket problem prøver programmet å løse?
- b) Hvilken metode er brukt til å løse problemet? Forklar kort detaljene i programmet.
- c) Du skal nå bestemme $y(0.5)$ med 2 steg og med 5 steg.
 - i) Utfør beregningene for hånd med 2 steg ($h = 0.25$). Vis utregningene og sammenlign med programmets utskrift.
 - ii) Utfør beregningene for hånd med 5 steg ($h = 0.1$). Vis utregningene i en tabell.
- d) Sammenlign de numeriske verdiene for $y(0.5)$ fra de to kjøringene med den eksakte løsningen. Hva kan du si om nøyaktigheten?

1.4 Eulers metode for systemer (ikke vektlagt)

I denne seksjonen skal vi se på hvordan vi kan løse systemer av differensialligninger ved hjelp av Eulers metode. Et system av differensialligninger kan for eksempel se slik ut:

$$\begin{aligned}y' &= f(t, y, z, w), \\z' &= g(t, y, z, w), \\w' &= h(t, y, z, w).\end{aligned}$$

med initialbetingelsene $y(t_0) = y_0$ og $z(t_0) = z_0$. For å løse dette systemet ved hjelp av Eulers metode, starter vi, på samme måte som i seksjon 1.3, med å definere en steglengde, Δt , og setter $t_n = t_0 + n\Delta t$ (legg merke til at vi her kaller den uavhengige variabelen for t i stedet for x . Dette har ingen betydning, og vi kunne like gjerne brukt x). Vi ønsker å finne tilnærmede verdier for $y(t_n)$, $z(t_n)$ og w_n , og kaller disse y_n , z_n og w_n . Vi regner ut neste tilnærmingsverdiene ved å bruke en lineær approksimasjon slik som tidligere. Altså, de neste tilnærmingsverdiene kan beregnes ved hjelp av

$$\begin{aligned}y_{n+1} &= y_n + f(t_n, y_n, z_n, w_n)\Delta t, \\z_{n+1} &= z_n + g(t_n, y_n, z_n, w_n)\Delta t. \\w_{n+1} &= w_n + h(t_n, y_n, z_n, w_n)\Delta t.\end{aligned}$$

Eksempel 1.4.1: La oss se på systemet

$$\begin{aligned}y' &= 2yt - 4\cos(z) + 2, \\z' &= te^{-y} + yz, \\w' &= 0\end{aligned}\quad \diamond$$

med initialbetingelsene $y(0) = -2$, $z(0) = 5$ og $w(0) = 1$ ved hjelp av Eulersmetode i intervallet $t \in [0, 1]$. Vi skal skrive et program som skisserer løsningskurvene i intervallet og regne ut $y(1)$ og $z(1)$. La $\Delta t = 0,001$.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(t,y,z,w):
5     return 2*y*t - 4*np.cos(z) + 2
6 def g(t,y,z,w):
7     return np.exp(-y)*t + y*z
8 def h(t,y,z,w):
9     return 0
10
11 tstart = 0
12 tslutt = 1
13
14 h = 0.001
15 N = int((tslutt - tstart)/h)
16
17 t = np.zeros(N+1)
18 for i in range(N+1):
19     t[i] = tstart + i*h
20
21 # Initialverdier
22 y = np.zeros(N+1)
23 z = np.zeros(N+1)
24 w = np.zeros(N+1)
25 y[0] = -2
26 z[0] = 5
```

```

27 w[0] = 1
28
29 # Eulers metode for system
30 for i in range(N):
31     y[i+1] = y[i] + f(t[i], y[i], z[i], w[i]) * h
32     z[i+1] = z[i] + g(t[i], y[i], z[i], w[i]) * h
33     w[i+1] = w[i] + h(t[i], y[i], z[i], w[i]) * h
34
35 # Tegner grafen
36 plt.grid(ls='--')
37 plt.plot(t, y, label='y')
38 plt.plot(t, z, label='z')
39 plt.plot(t, w, label='w')
40 plt.legend()
41
42 print("=" * 45)
43 print(f"\nDen numeriske løsningen gir:")
44 print(f"y({tslutt}) = {y[N]:.6f}")
45 print(f"z({tslutt}) = {z[N]:.6f}")
46 print(f"w({tslutt}) = {w[N]:.6f}")
47 print(f"Antall steg brukt: {N}")
48 print(f"Steglengde h: {h:.4f}")

```

Med å kjøre programmet får man:

```

1 =====
2
3 Den numeriske løsningen gir:
4 y(1) = 3.242637
5 z(1) = 8.630425
6 w(1) = 1.000000
7 Antall steg brukt: 1000
8 Steglengde h: 0.0010

```

Oppg. 1.4.1: a) Skriv et program for å løse systemet

$$\begin{aligned}
 y' &= 2y - z + 3, \\
 z' &= y + 2z.
 \end{aligned}$$

med initialbetingelsene $y(0) = -1$ og $z(0) = 1$ ved hjelp av Eulersmetoden i intervallet $t \in [0, 1]$. Programmet skal skissere løsningskurvene i intervallet og regne ut $y(1)$ og $z(1)$. La $\Delta t = 0,001$.

b) Utvid programmet slik at den kan løse:

$$\begin{aligned}
 y' &= 2y - z + w + 3, \\
 z' &= y + 2z - w, \\
 w' &= 3y - 2z + 2w
 \end{aligned}$$

med initialbetingelsene $y(0) = -1$ og $z(0) = 1$ og $w(0) = 2$, og skisser løsningskurvene i intervallet $t \in [0, 1]$.

odeint-funksjonen for å løse ordinære differensialligninger (ikke vektlagt)

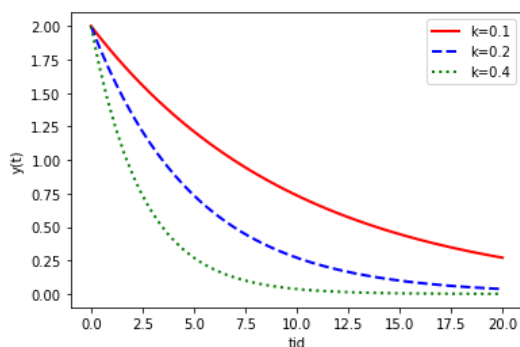
Funksjonen `odeint (model, y0,t )` kan brukes til å løse ordinære differensialligninger der $\frac{dy}{dt}$ er gitt i en funksjon, `dydt = model(y,t)`:

```
1 def model(y,t,k):
2     dydt = f(y)
3     return dydt
```

y_0 er startverdien ($y(t_0) = y_0$).

Eksempel 1.4.2: Skriv et program som løser differensialligningen $\frac{dy}{dt} = -ky^2$, og tegner løsningskurven for $k = 0.1, 0.2, 0.4$ gitt $y(0) = 2$.

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # funksjonen setter opp dy/dt
6 def model(y,t,k):
7     dydt = -k * y**2
8     return dydt
9
10 # startverdi
11 y0 = 2
12 # tidslinjen
13 t = np.linspace(0,20)
14
15 # Løser ODEs
16 k = 0.1
17 y1 = odeint(model,y0,t,args=(k,))
18 k = 0.2
19 y2 = odeint(model,y0,t,args=(k,))
20 k = 0.4
21 y3 = odeint(model,y0,t,args=(k,))
22
23 # Tegner løsningskurvene
24 plt.plot(t,y1,'r-',linewidth=2,label='k=0.1')
25 plt.plot(t,y2,'b--',linewidth=2,label='k=0.2')
26 plt.plot(t,y3,'g:',linewidth=2,label='k=0.4')
27
28 plt.xlabel('tid')
29 plt.ylabel('y(t)')
30 plt.legend()
31 plt.show()
```



1.5 Oppgaver og programmeringsoppgaver i kapittel 9

Oppg. 1.5.1: Skriv et program som løser differensialligningen $\frac{dc}{dt} = -kc^2$, og tegner løsningskurven for $k = 0.1, 0.2, 0.4$ gitt $c(0) = 5$.

Oppg. 1.5.2: Skriv et program som løser differensialligningen $\frac{dV}{dt} = kV^{(2/3)}$, og tegner løsningskurven for $k = 0.1, 0.3, 0.5$ gitt $V(0) = 2$.

Oppg. 1.5.3: Betrakt startverdi problemet

$$\frac{dy}{dt} = 3te^{-y}, \quad y(0) = 2,$$

- Bruk Eulers metode med $h = 0.2$ til å bestemme $y(0.6)$ ved å bruke $h = 0.2$.
- Løs startverdi problemet ved å løse differensialligningen, og sammenligne svaret med a).
- Skriv et program som bruker Eulers metode med $h = 0.2$ til å løse startverdi problemet, og som skriver ut løsningen i en tabell der man ser $x_{n-1}, y_{n-1}, dy/dx$ og (x_n, y_n) for hvert steg.

Oppg. 1.5.4: Et program er gitt som:

```
1 # DE1 1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def f(x,y):
6     return 0.5*y
7
8 xstart = 0
9 xslutt = 10
10
11 h = 2.5
12 N = 4
13
14 x = np.zeros(N+1)
15 for i in range(N+1):
16     x[i] = xstart + i*h
17
18 y = np.zeros(N+1)
19 y[0] = 1
20
21 print('\tx[n]\ty[n]\t dy/dx \ty[n+1]')
22
23 for i in range(N):
24     y[i+1] = y[i] + f(x[i], y[i]) * h
25     print(f"n={i}   x[{i}]= {x[i]:7.4f}   y[{i}]= {y[i]:7.4f}   y[{i+1}]= {y[i+1]:7.4f}")
26
27 # DE1 2
28
29 plt.plot(x,y,'--',label='Numerisk løsning')
30 plt.xlabel('x')
31 plt.ylabel('y')
32
33 def yeksakt(x):
34     return np.exp(0.5*x)
35
36 xpen = np.linspace(xstart,xslutt,100)
37 plt.plot(xpen,yeksakt(xpen),label='Den eksakte løsningen')
38 plt.legend()
39
```

```

40 print("=" * 45)
41 print(f"\nDen numeriske løsningen gir at y({xslutt}) = {y[N]:.6f}")
42 print(f"Antall steg brukt: {N}")
43 print(f"Steglengde h: {h:.4f}")
44 print(f"Den eksakte løsningen er at y({xslutt}) = {yeksakt(x[N]):.6f}")

```

a) Hvilket problem prøver programmet å løse?

b) Hvilken metode er brukt til å løse problemet?

Skriv kort om detaljene i programmet. Her kan du forklare den delen av programmet merket DEL 1 linje for linje (du trenger ikke å beskrive/forklare DEL 2).

c) Ved å kjøre programmet får man:

```

1      x[n]      y[n]      dy/dx  y[n+1]
2 n=0      x[0]= 0.0000  y[0]= 1.0000  y[1]= 2.2500
3 n=1      x[1]= 2.5000  y[1]= 2.2500  y[2]= 5.0625
4 n=2      x[2]= 5.0000  y[2]= 5.0625  y[3]=11.3906
5 n=3      x[3]= 7.5000  y[3]=11.3906  y[4]=25.6289
6 =====
7
8 Den numeriske løsningen gir at y(10) = 25.628906
9 Antall steg brukt: 4
10 Steglengde h: 2.5000
11 Den eksakte løsningen er at y(10) = 148.413159

```

i) Hva er den numeriske verdien for $y(10)$?

ii) Regn ut numerisk ved hjelp av Eulers metode $y(2.5)$ og $y(5)$, og sjekk svarene du får med utskriften fra programmet (utregning skal vises).

d) Hva kan endres i programmet for å øke presisjonen i programmet?

Oppg. 1.5.5: Et program er laget for å løse et startverdiproblem numerisk:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(t, y):
5     return 0.02 * y
6
7 t0 = 0
8 y0 = 2
9 T = 0.6
10
11 hVerdier = [0.4, 0.2, 0.1]
12
13 print("=" * 70)
14 print(f"Startverdiproblem: dy/dt = 0.02*y, y({t0}) = {y0}")
15 print(f"Beregner y({T}) med Eulers metode for ulike steglengder h:")
16 print("=" * 70)
17
18 for h in hVerdier:
19     N = int(np.round((T - t0) / h))
20     t = np.zeros(N+1)
21     y = np.zeros(N+1)
22     t[0] = t0
23     y[0] = y0
24
25     print(f"\n\n--- Steglengde h = {h} (N = {N} steg) ---")
26     print('\t\t t[n]\t\t\t y[n]\t dy/dt \t\t y[n+1]')
27     print('-' * 60)
28
29     for i in range(N):

```

```

30     y[i+1] = y[i] + f(t[i], y[i]) * h
31     t[i+1] = t[i] + h
32     print(f"n={i}    t[{i}]= {t[i]:7.4f}    y[{i}]= {y[i]:7.4f}    {f(t[i], y[i])}:6.4f}    y[{i+1}]= {y[i+1]:7.4f}")
33
34     yEksaktVerdi = 2 * np.exp(0.02 * T)
35     feil = abs(yEksaktVerdi - y[N])
36     print('-', * 60)
37     print(f"\nNumerisk løsning: y({T}) = {y[N]:.8f}")
38     print(f"Eksakt løsning:    y({T}) = {yEksaktVerdi:.8f}")
39     print(f"Feil: {feil:.2e}")
40     print("=" * 70)
41
42 tEksakt = np.linspace(t0, T, 200)
43 yEksaktVerdier = 2 * np.exp(0.02 * tEksakt)
44
45 plt.figure(figsize=(10, 6))
46 plt.plot(tEksakt, yEksaktVerdier, 'k-', label='Eksakt løsning', linewidth=2)
47
48 for h in hVerdier:
49     N = int(np.round((T - t0) / h))
50     t = np.zeros(N+1)
51     y = np.zeros(N+1)
52     t[0] = t0
53     y[0] = y0
54
55     for i in range(N):
56         y[i+1] = y[i] + f(t[i], y[i]) * h
57         t[i+1] = t[i] + h
58
59     plt.plot(t, y, 'o--', label=f'h = {h}', markersize=4)
60
61 plt.xlabel('t')
62 plt.ylabel('y(t)')
63 plt.title('Sammenligning av Eulers metode med ulike steglengder')
64 plt.legend()
65 plt.grid(True)
66 plt.show()
67
68 print("\nVi ser at feilen minker etter hvert som steglengden h blir mindre.")

```

- a) Hvilket startverdiproblem forsøker programmet å løse numerisk?
- b) Hvilken numerisk metode er implementert? Forklar kort prinsippet for metoden.
- c) Forklar hva programmet utfører: Gå gjennom koden og forklar hva de ulike delene gjør.
- d) I programmet testes tre ulike steglengder: $h = 0.4$, $h = 0.2$ og $h = 0.1$.
 - a) Utfør (for hånd) de tre første iterasjonene av Eulers metode for $h = 0.2$ for å finne tilnærminger til $y(0.2)$, $y(0.4)$ og $y(0.6)$. Sammenlign med verdiene programmet gir.
 - b) Hva forteller utskriften fra programmet om sammenhengen mellom steglengden h og feilen i estimatet for $y(0.6)$?
- e)
 - i) Endre programmet slik at det i tillegg til $y(0.6)$ også beregner $y(1.2)$ med de samme steglengdene $h = 0.4, 0.2, 0.1$. Kjør programmet og noter feilen for hver steglengde.
 - ii) Forklar hvorfor feilen for $y(1.2)$ er større enn for $y(0.6)$ for den samme steglengden h .

Blandede oppgaver

Oppg. 1.5.6: Løs startverdiproblemet, og sett opp et program som løser startverdiproblemet, og skisserer løsningskurven i samme koordinatsystem: $\frac{dy}{dt} = -ay^2 + by$, $y(0) = 5$ når: .

- a) $a = 0.1$ og $b = 0.2$
- b) $a = 0.1$ og $b = 0.01$.
- c) $a = 0.01$ og $b = 0.2$.

Oppg. 1.5.7: Skriv et program som løser differensiallingen og tegner løsningskurvene til startverdiproblemer:

$$\frac{dy}{dt} = -2ty, y(0) = 2$$

$$\frac{dy}{dt} = -2ty^2, y(0) = 2$$

Oppg. 1.5.8: Skriv et program som løser differensiallingen og tegner løsningskurven til startverdiproblemet:

$$x'' + 0.4x' + 0.05x = \cos(2t), x(0) = 1, x'(0) = 0$$

Hint: Skriv differensiallingen som et lineært differensialligningssystem og bruk Eulers metode.

Kapittel 2

Litt programmering med Python

Python er et objektorientert programmeringsspråk, som ble laget av Guido van Rossum på 80-tallet, og ble først presentert i 1991. Python er et generell programmeringsspråk som kan brukes på alle moderne operativsystemer. Python har en god del fordeler, blant annet er det enkelt å bruke og har mange innebygde funksjoner. Python har en enkel syntaks som ligner på det engelske språket. Python er et ganske moderne programmeringsspråk som kan brukes i forskjellige vitenskapelig kontekster (f.eks. i maskinlæring). Den kan brukes til behandling av tekst, tall, bilder, vitenskapelige data og omtrent alt annet du kan bruke på en datamaskin. Den brukes daglig i driften av f.eks. Googles søkemotor, videodeling YouTube, NASA og New York Børs.

2.1 Å komme i gang med Python

Det er to hovedversjoner av Python, Python 2 og Python 3. Disse versjonene er ganske forskjellige. Her bruker vi Python 3, fordi den er mer semantisk korrekt og støtter nyere funksjoner.

Python er et tekstbasert programmeringsspråk, og for å komme igang med å programmere, så har man flere alternativer blant annet:

- Programmere på nett: Det er flere tjenester på nettet som lar deg skrive og kjøre python-programmer, som for eksempel:

<https://trinket.io/python3>

<https://replit.com/new/python3>

- Installere en kildekode-plattform som både er en kode-editor, men som har innebygget en egen «Python-motor».

<https://www.spyder-ide.org/>

<https://jupyter.org/>

Eksempel på et slikt program er Spyder. Dette er en åpen kildekode-plattform integrert i utviklingsmiljø for vitenskapelig programmering med Python. Spyder integreres med en rekke fremtredende pakker i den vitenskapelige Python-pakken, inkludert numpy, scipy, Matplotlib, pandas, IPython, sympy og Cython, samt annen programvare med åpen kildekode. Det finnes også andre slike program:

<https://www.anaconda.com/>

<https://codewith.mu/en/download>

Konsollvinduet og script i editorvinduet

I dette heftet har vi valgt å bruke Spyder. Vinduet til venstre er editorvinduet og nederst på høyre side er konsollvinduet. I Spyder kan man kjøre Python-kode direkte i konsollvinduet. Dette passer for små og enkle beregninger, for eksempel hvis vi bare skal regne ut et regnestykke som $2 + 5$,

```
1 In[1]: 2+5
2 Out[2]: 7
```

Men når man skriver lengre programmer, er det enklere å skrive koden i et *script*. Et Python-script er en kjørbart fil som inneholder Python-kode. Det vil si at hvis man skriver inn $2+5$ i et script, så skjer det ikke noe før man kjører scriptet. Her er et eksempel på et script som skal beregne arealet til et rektangel:

```
1 a = 5
2 b = 3
3 # Regner ut de to rottene
4 A= a*b
5 print('Arealet til et rektangel med sidelengdene', 'a =', a, 'og b=', b, 'er
    lik', A)
```

Dersom vi kjører dette scriptet, vil vi få:

Arealet til et rektangel med sidelengdene a = 5 og b= 3 er lik 15

online-programmering

Det er anbefalt av flere læresteder å bruke:

<https://trinket.io/python3>

når man skal programmere på nettet.

2.2 Noen viktige elementer i Python-programmering

De viktige elementene i Python-scriptet er:

- *Kommentarer*: Når man programmerer, så lønner det seg å forklare hva koden gjør. Det gjør man med tegnet `#` eller `' '` eller `" "`. Dette tegnet tolker Python som «resten av denne linjen skal ikke leses av programmet». Kommentarer kan også settes mellom enten 'enkle apostrofer' eller doble apostrofer".
- *Streng*: En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.
- *Funksjoner*: en gruppe kodelinjer, som utfører en spesifikk oppgave, for eksempel beregner en matematisk uttrykk eller skriver ut en tekst. En funksjon kan brukes flere steder i ett program, med å kalle funksjonen.
- *Pakker*: For å utføre beregninger, er det nødvendig å importere en passende pakke. Her importerer vi pakken `numpy` som `np`, slik at vi kan bruke for eksempel funksjonen `sqrt` som tilhører pakken som `np.sqrt`.
- *Lister*: Når vi trenger å lagre flere verdier i samme variabel, kan vi benytte lister. Lister er samlinger med tall, strenger, variabler eller annet.
- *Dat typer*: En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype, som behandles (tekst (`str`), tall (`int`, `float`), vektor (`array`)).
- *Variabler*: Videre definerer vi de tre koeffisientene a, b og c . I Python kalles disse *variabler*, og likhetstegnet angir verdien deres. Også x_1 og x_2 som vi så regner ut, er variabler, og vi ser at i utregningen brukes a, b og c , som har verdiene vi allerede har gitt dem.
- *Indeksering*: Indeksering er prosessen som ordner et sett med data slik at det blir mulig å spore opp datene. Indeksering i Python starter ved indeksen 0.
- *Løkker*: Løkker brukes for å gjenta noe mange ganger etter hverandre

2.3 Noen av Pythons innebygde funksjoner

Python-interpreter har en rekke innebygde funksjoner som alltid kan kalles med å skrive deres navn med riktige input som inngår i funksjonene.

Funksjon	Beskrivelse
<code>help()</code>	Kaller det innebygde hjelpesystemet
<code>print()</code>	Skriver ut data
<code>abs()</code>	Returnerer absoluttverdi av et tall
<code>float()</code>	Returnerer et flyttall
<code>int()</code>	Returnerer et heltall
<code>input()</code>	For å få input fra bruker
<code>input(float())</code>	For å få input (et reelt tall) fra bruker
<code>input(int())</code>	For å få input (et heltall) fra bruker
<code>input(str())</code>	For å få input (tekst) fra bruker
<code>len()</code>	Returnerer lengde av et objekt
<code>list()</code>	Returnerer en liste
<code>range()</code>	Returnerer en sekvens av hele tall, som standard starter fra 0 og inkrementerer tallene med 1
<code>round()</code>	Returnerer rundverdi av et tall

2.4 Grunnleggende Python-programmering

Kommentere i koden med ' ' og

Kommentarer (en forklarende tekst) er det første vi vanligvis møter i koden. En kommentar starter enten med en hash, og varer ut linjen, eller skrives inne i triple anførselstegn, som første linje i eksempelkoden over. Ved triple anførselstegn kan kommentaren vare over flere linjer. Dette er tekst eller kode som ignoreres når koden kjøres. Det er lurt å ha med kommentarer underveis i koden, fordi det gjør den enklere å lese, og det kan være til hjelp når du senere ser tilbake på den.

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Fri Dec 16 12:39:51 2022
4 @author: ahas
5 """
6 print('Skriv ut noe')
```

Ved å kjøre programmet får vi:

```
1 Skriv ut noe
```

Strenger

En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.

Man kan skrive tekst (strenger) både med apostrof ('... ') og med anførselstegn ("..."):

```
1 print('Denne linjen blir skrevet ut')
2 print('Denne linjen blir skrevet ut')
```

Man får ikke feilmelding ved å bruke ekstra mellomrom:

```
1 print('Denne linjen blir skrevet ut')
2 print('Denne linjen blir skrevet ut')
```

Man får feilmelding hvis man glemmer noe av syntaksen man skal ta med:

```
1 print(Denne linjen blir skrevet ut')
```

Vi skal se nærmere i notatene at print-funksjonens tolker følgende som streng og som tall:

```
1 print('3+4')
2 print(3+4)
```

Resultatet av den første programsetningen blir 3+4 og den andre 7.

Oppg. 2.4.1: a) Skriv en programkode som skriver ut: Denne setningen skrives ut!

b) Finn feilen i følgende programsetninger:

```
1 Print('Jeg liker programmering')
2 print('Jeg liker programmering!')
```

Oppg. 2.4.2: Hva blir resultat av følgende programsetninger:

a) `print('5+7')`

b) `print(5+7)`

c) `print('5+7')`

d) `Print('5+7')`

e) `print('5+7')`
`print()`

Kalkulator

Vi ønsker at kalkulatoren skal kunne addere (+), subtrahere (-), multiplisere (*) og dividere (/). Vi kaller +, -, * og / for operatorer.

Matematiske operatorer

Operator	Utfører	Eksempel	Resultat
+	Addisjon	$5 + 2$	7
-	Subtraksjon	$5 - 2$	3
*	Multiplikasjon	$5 * 2$	10
/	Divisjon	$5 / 2$	2.5
//	Heltalldivisjon	$13 // 3$	4
**	EkspONENT	$2 ** 3$	8
%	Modulo	$13 \div 4$	1

- a/b gir kvotienten når divisoren a deles på dividenden b .
- $a//b$ gir antall heltall som er i kvotienten (når man deler a på b) når divisoren a deles på dividenden b .
- $a\%b$ gir restleddet når divisoren a deles på dividenden b .

Heltalldivisjon er divisjon uten restledd. Dette betyr at for eksempel både $14//7$ og $15//7$ blir 2, siden $14 : 7 = 2$ og $15 : 7 = 2 + \frac{1}{7}$.

- $a \div b$ (modulo) gir restleddet når man deler a på b .

Eksempel 2.4.1: Her er et eksempel fra konsollvinduet:

```
1 > 2+3+4*5
2 25
3 >3/2
4 1.5
5 >2**3 +1
6 9
7 > 15/7
8 2.142857142857143
9 >15//7
10 2
```

Oppg. 2.4.3: Regn ut disse med Python

a) $3 \cdot 5 + \frac{4}{5}$

b) $2^3 + 3^2 + \frac{1.25}{5}$

c) Hva resultatet til $18\%4$ blir?

Oppg. 2.4.4: Løs disse oppgavene v.h.a. Python

a) Regn ut arealet til en trekant med høyden 5 cm og grunnlinjen 8 cm.

b) Regn ut arealet til et parallelogram som har høyden 7 cm og grunnlinjen 10 cm.

Arealet til et parallelogram er altså lik lengden (grunnlinjen) multiplisert med høyden.

c) Regn ut arealet til en trapes som har høyden 6 cm og lengden til de to parallelle sidene er 8 cm og 12 cm.

Arealet av et trapes regnes ut ved å multiplisere høyden og den gjennomsnittlige lengden av de to parallelle sidene.

Å hente π fra numby-biblioteket

Eksempel 2.4.2: Vi ønsker å bestemme arealet til en sirkel med radius 5 cm.

Her har vi to muligheter Vi kan regne direkte med π som verdien 3.14, og regne direkte på kommandolinjen:

```
1 > 3.14*5**2
2 78.5
```

eller vi kan definere pi som 3.14 og regne ut arealet et skript:

```
1 pi=3.14
2 r=5
3 Areal = pi*r**2
4 print(Areal)
```

Ved å kjøre programkoden får man 78.5.

Vi lærer senere at vi kan importere pi fra *math*- eller *numpy*-biblioteket, der man enten installer hele *math*- eller *numpy*-biblioteket eller bare installerer det man trenger. Du kan bruke *numpy*-bibliotekets *numpy.pi* eller *math*.bibliotekets for *math.pi* og få verdien av den matematiske konstanten π i python.

```
1 import numpy
2 r=5
3 Areal = numpy.pi*r**2
4 print(Areal)
```

```
1 import numpy as np
2
3 r=5
4 Areal = np.pi*r**2
5 print(Areal)
```

Hvis man skal regne ut med kjente verdier som π eller kjente funksjoner som cos, kan man enten installere hele *numpy*-biblioteket:

```
1 from numpy import *
2 r=5
3 Areal = pi*r**2
4 print(Areal)
```

eller bare innstallere det man trenger:

```
1 from numpy import pi
2
3 r=5
4 Areal = pi*r**2
5 print(Areal)
```

Hvis man importerer fra numpy bare det man trenger, kan bare bruke dem uten å bruke np. Tilsvarende gjelder dette også for funksjoner som vises i det neste eksempelet.

2.5 Datatyper

En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype som behandles, og hvilke regler som skal brukes for hvordan dataene skal behandles. Her har vi fokus bare på enkle datatyper, datatyper som bare inneholder 'en verdi. De mest brukte er:

- int - integer (heltall som 1,2,3)
- float - desimaltall (eks 2.5 - OBS: her må man bruke punktum og ikke komma)
- str (string) - tekst, OBS: skrives mellom anførselstegn
- bool - boolean (True/False - OBS: når man skal gi en variabel verdien True eller False, må de skrives med stor forbokstav)

I tillegg har vi noen sammensatte datatyper, hvor hver datatype kan inneholde flere verdier, f.eks.

- list - lister
- tuple - tupler/tuppel (ligner på lister, men er en ikke-muterbar sekvens - den kan ikke endres og brukes først og fremst på data som ikke skal endres, slik som ukedager og måneder)
- set - mengder
- dict - dictionary (oppslagsliste)

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	{1, 3, 5}
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Eksempel 2.5.1: Her vises et eksempel, hvordan strenger oppfører seg med matematiske operasjoner:

```
1 print('tre' + 'ti')
2 print('to' * 3)
```

Utskriften blir da:

```
1 treti
2 tototo
```

2.6 Variabler

Som i matematikk brukes variabler i programmering. Variabler brukes til å huske data til bruk senere i programmet, for eksempel til utregninger, beslutninger og for å gjøre endringer på informasjon. Variabler kan innta verdier eller tekst, og for tekst skal ha med apostrof ('... ') eller anførselstegn ("...") rundt seg. Man kan opprette en variabel i et program ved å skrive en bokstav eller en tekst ettefulgt av et liketstegn og

- verdien som variabelen skal innta hvis variabelen skal være *et tall*
- apostrof ('... ') eller anførselstegn ("...") rundt seg (åpnes og avsluttes) hvis variabel skal være *tekst/streng*.

Som vist nedenfor her:

```
1 x= 2
2 y= 'Tekst'
3 z= "Tekst"
```

I løkkene blir ofte variablene oppdatert ved å legge til et tall til variabelen eller multiplisere variabelen med et tall, for eksempel:

Eksempel 2.6.1: Her settes variabelen n til verdien 2 og blir deretter oppdatert:

```
1 n = 2
2 # det legges til 4
3 n = n+4
4 # Resultatet ganges med 5
5 n=5*n
6 # Resultatet legges til 4
7 n += 4
8 # Resultatet ganges med 5
9 n* = 5
10
11 print('n blir da',n)
```

Her blir verdien til variabelen n satt sammen med 4 og ganget med 5 på to ulike måter, før resultatet blir multiplisert med 5 og blir vist. Svaret er da $((2 + 4) * 5 + 4) * 5 = 170$. ◇

Oppg. 2.6.1: Skriv en programkode som utfører følgende regneoperasjoner ved hjelp av variabler:

- Skriv en programkode som leser inn to tall m og n som tar verdiene 7 og 1 henholdsvis, deretter blir m summert med 3 og n ganges med 5, og til slutt multipliseres m og n sammen og resultatet blir skrevet ut på skjermen som: *Produktet er* og resultatet.
- Skriv en programkode som leser inn to tall a og b som tar tallet 5 og teksten *TALL*, deretter multipliseres a og b med henholdsvis 2 og 3 og de oppdaterte variabelene a og b blir vist på skjermen

Oppg. 2.6.2: Skriv en programkode som beregner volum og overflateareal til en rektangulær rett prisme med lengde 5 meter, bredde 4 meter og høyde 3 meter.

Oppg. 2.6.3: Skrive en programkode som regner ut $y = h - \frac{1}{2}gt^2 + v_0t$ for $h = 3.5$, $g = 9.81$, $v_0 = 20$ og $t = 5$, og skriver på skjermen.

Avrunding

F-literalene bruker vi når vi vil runde av et tall i en *print-kommando*. Rett før høyre mengdeparentes legger vi inn et kolon. Da kan vi styre hvordan utskriften formateres. Vi runder av til to desimaler med formatdefinisjonen «.3f». Her betyr bokstaven «f» at variabelen skal skrives ut med et fiksert antall desimaler, og tallet betyr at det skal være tre desimaler. Hvis du i stedet skriver «.2f», får du to desimaler.

Eksempel 2.6.2: Her er et eksempel, hvordan to tall og deres sum blir avrundet:

```

1 a = 25/7
2 b = 32
3 print(f"Her er a {a:.3f} og b {b:.1f}. Summen er {a+b:.3f}.")

```

Det er vanlig at resultatet fra programmet som skrives til skjermen, inneholder både tekst og verdier av variabler. I programmet bruker vi det som kalles f-literal. Vi setter en «f» rett etter venstreparentesen for å fortelle at det kommer en f-literal. Alt som står inni mengdeparenteser i en f-literal, blir tolket som Python-kode. Det vil si at a blir tolket som verdien av variabelen a.

2.7 Å sende inn datasett i Python-program

1. I dataprogram kan variablene deklarerer med å definere variablene som får inndelt en verdi:

```

1 r=2
2 n=5

```

2. Et dataprogram kan ta imot data fra brukeren hver gang man prøver å kjøre programmet:

```

1 alder=input("Skriv inn alderen din:")
2 print(f''Du er {alder} år.'')

```

Her kunne man angi alderen som tall eller tekst. Hvis man ønsker seg å presisere hvilken datatype (heltall, flyttall og tekst), vil man ta i bruk videre i programmet følgende:

```

1 alder=int(input("Skriv inn alderen din:"))
2 r = float(input("Skriv inn radiusen til sirkelen (et reelt tall) her: "))
3 n = int(input("Skriv inn a (et heltall) her: "))

```

3. Når vi skal behandle datafil i Python, er det første vi må gjøre å laste ned en fil med dataene og lagre den i et hensiktsmessig format. De vanlige formatene er .txt og .csv. Når vi laster ned filer med stort tallmateriale, er det ofte lurt å bruke .csv (se avsnitt 0.14).

I neste eksempel vises hvordan `float(input())` og `float(input())` for å lese inn takk og hvordan `print(f("tekst og {tall}"))` kan brukes for å formatere det som skrives ut på skjermen.

Formatering av tekst og tall

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra brukeren, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```

1 print('Her leses inn variablene')
2 a = int(input("Skriv inn a (et heltall) her: "))
3 b = float(input("Skriv inn b (et reelt tall) her: "))
4
5 c= a+b
6 print()
7 print(f'Summen til {a} og {b} er: {c} ')
8 print() # lager en linje mellomrom
9 print(f'Summen til {a} og {b} er:c=', c)
10 print()
11 print('formatering:')
12 print(f'Her er a {a:.2f} og b {b:.1f}. Summen er {c:.3f}.')
13 print(f'Summen til {a} og {b} er: {c:.3e} ')

```

Ved å kjøre programmet får vi da på skjermen

```

1 # Her leses inn variablene
2
3 Skriv inn a (et heltall) her: 11
4 Skriv inn b (et reelt tall) her: 2.35
5
6 Summen til 11 og 2.35 er: 13.35

```

```
7 Summen til 11 og 2.35 er: c =13.35
8
9 formatering:
10 Her er a 11.00 og b 2.4. Summen er 13.350.
11 Summen til 11 og 2.35 er: 1.335e+01
```

Hvis man skulle lese tekst fra brukeren, kunne man bruke `str(input())`.

Oppg. 2.7.1: Skrive en programkode som oppretter tre variabler a , b og c der a blir gitt som input av brukeren, og $b = 3$ og $c = 5$. Programmet skal regne ut $d = \frac{1}{3}ab + c^2$, og skrive resultatet på skjermen.

Oppg. 2.7.2: Skrive en programkode som oppretter to variabler r og h som blir gitt som input av brukeren (radius og høyden til en rett sylinder), og som regner ut $V = \pi r^2 h$ (volumet til sylindere, $\pi = 3.14$), og skriver resultatet på skjermen.

2.8 Importering av pakker (numpy, math, pylab, sympy)

En pakke eller et bibliotek i Python er en samling av moduler, hvor en modul er en fil som inneholder Python-definisjoner (funksjonsdefinisjoner), men inneholder ikke kall til. Modul kan være en en samling av funksjoner og klasser som kan gjenbrukes i mange ulike programmer. Når man bruker en modul fra en pakke, skrives: `pakke.modul`, henter modulen `modul` fra pakken `pakke`. Det finnes mange ulike pakker man kan benytte i Python, og vi skal hovedsakelig bruke pakkene `numpy` (numerisk Python) og `matplotlib` her.

Disse to kommandoene for å importere hele pakken er ekvivalente:

```
1 import numpy
```

```
1 from numpy import *
```

Eller man kan bare importere det man trenger:

```
1 from numpy import pi
```

Noen foretrekker å bruke:

```
1 import numpy as np
```

der for eksempel for å bruke π skriver man i programmet `np.pi`.

Under følger et kort eksempel der vi importerer noen pakker, og bruker dem til en liten beregning av $y(x)$ for en gitt x -verdi.

Eksempel 2.8.1: Vi ønsker å beregne $y(x) = e^{-\pi x}$ for $x = 0.2$.

```
1 import numpy as np
2 x= 0.2
3 y = np.exp(-np.pi*x)
4
5 print('y er lik',y)
```

Eksempel 2.8.2: Man kan importere hele pakken eller deler av pakken:

```
1 import numpy as np          # Importerer hele pakken som får navnet np
2 # Vi vil bruke pakkens cosinus- og pi-funksjoner:
3
4 print("sinus av ",np.pi," er ", np.sin(np.pi))
5 print() # oppretter ekstra linje
6
7 # Vi kan ogsaa bare importere enkeltfunksjoner. Da kan vi kalle paa
  funksjonene uten aa angi pakkens navn.
8
9 from numpy import sin, pi    # importerer vi bare delene vi trenger.
10
11 print("sinus av ",pi," er ", sin(pi))    # Merk at vi ikke trenger pakkens
  adresse.
```

Oppg. 2.8.1: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

Eksempel 2.8.3: Vi vil regne ut $\sin(1) + \cos(4) + e^{-2}$.

```
1 import numpy as np
2 svar=np.sin(1)+np.cos(4)+np.exp(-2)
3 print(svar)
1 from numpy import exp, sin, cos
2 svar=sin(1)+cos(4)+exp(4)
3 print(svar)
```

numpy eller *math*-biblioteket?

numpy-biblioteket inneholder flerdimensjonale array- og matrise-datastrukturer, mens *math*-biblioteket brukes til å utføre matematikk på skalardata, for eksempel trigonometriske beregninger. Hvis man skal regne ut med kjente funksjoner som $\cos$, kan man enten importere hele *numpy*-biblioteket eller bare importere det man trenger:

```
1 from numpy import exp, sin, cos, tan, arcsin, arccos, arctan, exp
2 svar=sin(1)+cos(4)+exp(4)
3 print(svar)
```

Husk at de trigonometriske funksjonene ($\sin$, $\cos$ og $\tan$) tar inn en vinkel i radianer, og inversfunksjonene ($\arcsin$, $\arccos$ og $\arctan$) returnerer en vinkel i radianer.

Oppg. 2.8.2: Regn ut disse v.h.a Python

- a) $\cos(2) + \tan(1.5)$
- b) $e^4 + \log(2) + \ln(0.1)$

pylab-biblioteket

I de nye matematikk-lærebøkene for videregående-trinnet brukes *pylab*-biblioteket, som har mange innbygde funksjoner. *pylab*-pakken inneholder både *numpy* og *matplotlib*.

```
1 from pylab import *
2 print(cos(1))
```

Noen foretrekker *numpy* og at man bruker

```
1 import numpy as np
```

der man for eksempel for å bruke $\sin$ -funksjonen skal skrive *np.sin(x)*. Fordi man har da et bedre oversikt hvor i programmet har brukt pakken.

Oppg. 2.8.3: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

numpy- eller *sympy*-biblioteket?

numpy-biblioteket gjør beregningsmatematikk, mens *sympy*-biblioteket gjør symbolsk matematikk. For eksempel for å derivere eller å integrere:

```
1 from sympy import *
2
3 x = Symbol('x')
4 svar1= diff(x**3, x)
5 svar2= integrate(svar1, x)
6 print(svar2)
```

Andre pakker (*Matplotlib*)

Vi vil ofte ha behov for å visualisere data. Da kan biblioteket *Matplotlib* hjelpe oss. Her skal vi bruke modulen *pyplot* (som er en del av biblioteket *Matplotlib*) for å tegne en graf:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-3,3, num = 101)
5 f = x**3-3*x
6
7 plt.plot(x,f, '--')
8 plt.grid()
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'])
12 plt.savefig('MinPythonFigMappe/eksplot.png')
```

2.9 Indeksering

Vi lager indeksmengder med kommandoen `range`. Kommandoen `range(2, 5)` betyr heltallene fra og med 2 til, men ikke med, 5 (til og med 4).

<code>range(1,n+1)</code>	alle hele tall fra og med 1 til n
<code>range(1,n+1,m)</code>	alle hele tall fra og med 1 til n med m steglengde
<code>range(1,n+1,m)</code>	alle hele tall fra og med 1 til n med m steglengde

Hvis du utelater første argument i `range`, antar Python at første argument er 0. Dersom du legger til et tredje argument, angir dette steglengden i intervallet.

Kommando	Verdi
<code>range(0, 5)</code>	0, 1, 2, 3, 4
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(2, 5)</code>	2, 3, 4
<code>range(0, 5, 2)</code>	0, 2, 4
<code>range(10, 5, -1)</code>	10, 9, 8, 7, 6

2.10 Løkker (betingelser og gjentagelser)

En løkke brukes for beregninger eller handlinger som skal gjentas flere ganger. Man kan velge et gitt antall ganger eller at det skal gjentas inntil et mål (noe) er nådd. Fordelen med å bruke en løkke er at vi slipper å skrive mange setninger med programkode. Da blir koden enklere å feilsøke og rette, men den blir også mer kompakt, noe som gjør at det tar kortere tid å kjøre gjennom programmet.

if/elif/else-løkke

En if/elif/else-setning brukes når vi ønsker å utføre en blokk med kode når den gitte tilstanden er sant eller usant. Her er et eksempel:

```
1 a = float(input("Skriv inn a (et reelt tall) her: "))
2 if a > 2:
3     print("a er mer enn 2")
4
5 elif a < 2:
6     print("a er mindre enn 2")
7
8 else:
9     print("a er lik 2")
```

Ved å kjøre programmet og taste inn forskjellige verdier for *a* får vi:

```
1 Skriv inn a (et reelt tall) her: 4
2 a er mer enn 2
3 Skriv inn a (et reelt tall) her: 1
4 a er mindre enn 2
5 Skriv inn a (et reelt tall) her: 2
6 a er lik 2
```

Oppg. 2.10.1: Hvis -7 skrives inn når følgende programkoden kjøres, hva blir resultatet?

```
1 print('Skriv et tall')
2 a = int(input())
3 x=a
4 if x < 0:
5     x = -x
6 print('Absluttverdien til',a,' er', x)
```

Oppg. 2.10.2: Skriv en programkode for følgende:

- Les et tall fra skjermen, og sjekk om tallet er positivt, negativt eller null, og skriv resultatet på skjermen.
- Les et tall fra skjermen, og sjekk om tallet er partall eller oddetall, og skriv resultatet på skjermen.
- Les to tall fra skjermen og si om disse tallene har samme eller motsatt fortegn.

while-løkke

While-løkker brukes i tilfeller hvor man kan si at løkka skal utføres så lenge noe er sant.

```
1 i=0
2 while i < 3:
3     print("i =", i)
4     i += 1
```

Ved å kjøre programmet får man:

```
1 i = 0
2 i = 1
3 i = 2
```

Andre ganger vil vi gjøre bestemte utregninger så lenge en betingelse vi har satt på forhånd, er oppfylt. Vi skal vise bruken av denne type løkke ved hjelp av et eksempel. Legg merke til at $\text{range}(1, n+1)$ er alle hele tall fra og med 1 til n .

Eksempel 2.10.1: Skriv et program som regner ut $a_2, a_3, \dots, a_{10}$ når: $a_1 = 1$ og $a_{n+1} = 0.25a_n + 2$. Hva konvergerer a_n når $n \rightarrow \infty$?

Løsning:

```
1 n = 1
2 for i in range(1,10):
3     0.25*n + 2
4     print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 2.666\dots$ når $n \rightarrow \infty$. $\diamond$

Eksempel 2.10.2: Skriv et program som regner ut $a_2, a_3, \dots, a_{10}$ når: $a_1 = 1$ og $a_{n+1} = \frac{3}{4 - a_n}$. Hva konvergerer a_n når $n \rightarrow \infty$?

Løsning:

```
1 n = 1
2 for i in range(1,10):
3     n = 3/(4-n)
4     print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 1.0$ når $n \rightarrow \infty$. $\diamond$

Eksempel 2.10.3: Betrakt summen $\sum_{n=1}^{\infty} \frac{5}{n}$.

Finn ut hvor mange ledd som trengs for å få summen over 20.

Løsning:

```
1 n = 1 # Definerer n og Sum ved start
2 Sum = 0
3 # Vil regne ut summeringen til summen er mindre eller lik 20
4 while Sum <= 15:
5     Sum += 5/n
6     n += 1
7     print('Summen passerer 15 etter', n, 'ledd. Da er summen', Sum)
```

Ved å kjøre programmet får man:

```
1 Summen passerer 15 etter 32 ledd. Da er summen 20.136225977182594
```

Oppg. 2.10.3: Skriv kort hva følgende programkode utfører:

```
1 innlest = input("Hva er 49+35?")
2 tall = int(innlest)
3
4 if tall != 84:
```

```

5 while tall != 84:
6     innlest = input("Regn igjen!")
7     tall = int(innlest)
8 else:
9     print("Du klarte det!")

```

Oppg. 2.10.4: Skriv en programkode som spør hva summen til $52+19$ er, og leser inn svaret fra brukeren, og hvis svaret er feil, skriver ut *Feil. Regn igjen!*, og hvis det er rett, skriver ut *Du klarte det!*

Oppg. 2.10.5: Følgende programkode spør leseren om å skrive et partall, og deretter sjekker om dette tallet er partall, og bestemmer summen til alle partall til og med dette tallet.

```

1 tall = int(input("Skriv inn et partall: "))
2
3 if tall%2 == 1:
4     print("Tallet er ikke et partall!")
5 else:
6     sum = 0
7     i=0
8     while(i <= tall):
9         sum = sum + i
10        i = i+2
11
12 print(sum)

```

Skriv en programkode som gjør dette for oddetall.

Oppg. 2.10.6: Vi vil prøve å finne det minste heltallet x (heltall) for at $3^x > 81$.

Følgende program vil telle hvor mange ganger, må gange 3 med seg selv for å få mer enn 81.

```

1 x0= 1
2 b = 81
3 x = 0
4 x1 = x0
5
6 while x1 <= b:
7     x += 1
8     x1*= 3
9
10 print(f"Svaret er {x}.")

```

Skriv programmet og sjekk om du får svaret 4.

Modifiser programmet for å gjøre dette for ligningen $4^x = 1024$

for-løkke

for-løkken kan brukes til å gjenta en kodeblokk et kjent/begrenset antall ganger, eller gå gjennom en sekvens, element for element:

```
1 n=2
2 for i in range(1,n+1):
3     b=i**2+1
4     print(b)
```

Ved å kjøre programmet får man:

```
1 2
2 5
```

Legg merke til at $\text{range}(1, n+1)$ betyr indeksmengde fra og med 1 til og med n (og ikke til og med $n + 1$).

Oppg. 2.10.7: Skriv hva resultatet av følgende programkode blir:

```
a) i=0
2 for i in range(2):
3     print('A')
4     for i in range(3):
5         print('B')
```

```
b) for x in [50, 60]:
2   for y in [1, 2, 3]:
3       print(x+y, end=' ')
```

```
c) for x in [50, 60]:
2   print()
3   for y in [1, 2, 3]:
4       print(x+y, end=' ')
```

Oppg. 2.10.8: Hva får man av følgende programkoder? Skriv følgende i et script og kjør programmet.

```
a) for i in range(0,10):
2   print(i, end=" ")
```

```
b) for i in range(1,10):
2   print(i, end=" ")
```

```
c) for i in range(0,10,2):
2   print(i, end=" ")
```

```
d) for i in range(10,1,-1):
2   print(i, end=" ")
```

Oppg. 2.10.9: Skriv en programkode for følgende: Det dukker på skjermen: *Hei, vil du fortsette med programmet?* Hvis svaret er nei, avsluttes programmet.

Oppg. 2.10.10: Skriv en programkode som regner ut summen til de første 5 leddene i en geometisk rekke med førsteledd 3 og kvotient 2, og skriver ut på skjermen summen.

Oppg. 2.10.11: Skriv kort hva følgende program utfører:

```
1 for i in range(1,10):
2     for k in range(1,5):
3         print(i,k)
```

Oppg. 2.10.12: Skriv en programkode som regner ut summen til de første 7 leddene i en aritmetisk rekke med førsteledd 5 og kvotient 3, og skriver ut summen på skjermen.

Oppg. 2.10.13: a) Skriv hva resultatet til følgende programkode blir:

```
1 for i in range(1,2):
2     for k in range (1,3):
3         print(i,2*k)
```

b) Skriv også hva `range(5)` sender inn i løkken.
Skriv hva resultatet til programkoden blir

```
1 for i in range(5):
2     if i == 3:
3         print(i, end=" ")
4     else:
5         print(2*i, end=" ")
```

c) Skriv også hva `while i < 4` sender inn i løkken.

```
1 i = 1
2 while i < 4:
3     print(i)
4     if i == 3:
5         break
6     i += 1
```

Hvordan ville du endre løkken i del c) hvis du ville bruke `range`-syntaksen?

d) `= 1`

```
2 while i < 5:
3     print(i)
4     i += 1
5 else:
6     print("i er ikke lenger mindre enn 5")
```

e) Hva skriver følgende programkode ut?

```
1 r = 4
2 for i in range(1, r + 1, 1):
3     for j in range(1, i + 1):
4         print(j, end=' ')
5     print()
```

f) Hva skriver følgende programkode ut?

```
1 n=4;
2 for i in range(n):
3     for j in range(i):
4         print ('o ', end=" ")
5     print(' ')
6
7 for i in range(n,0,-1):
8     for j in range(i):
9         print ('* ', end=" ")
10    print(' ')
```

g) Skriv en programkode som får følgende resultat:

```
1 *
2 * *
3 * * *
4 o o o o
5 o o o
6 o o
7 o
```

Oppg. 2.10.14: Følgende programkode

```
1 n = 4
2 k = 4
3 for i in range(0,n+1):
4     for j in range(k-i,0,-1):
5         print(j,end=' ')
6     print()
```

Skriver ut:

```
1 4 3 2 1
2 3 2 1
3 2 1
4 1
```

a) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 1 2
3 1 2 3
4 1 2 3 4
```

b) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 4 3 2 1
2 3 2 1
3 2 1
4 1
```

c) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 2 1
3 3 2 1
4 4 3 2 1
```

d) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 1 2
3 1 2 3
4 1 2 3 4
5 1 2 3 4 5
```

2.11 Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler. I programmering er funksjoner en måte å samle en beregning utført av en eller flere variabler i en sammensatt bolk, såkalte *funksjoner*, slik at det blir lettere å skrive, modifisere, lese koden, bruke andre steder i programmet eller eventuelt i andre program. Syntaksen for funksjoner er kontrollordet *def*, deretter navnet på funksjonen etterfulgt av to parenteser som inneholder variabel (variabler), som inngår funksjonen og et kolon (:).

```
1 def funksjonsnavn(variabler):
2     kodeblokk
```

Husk å trykke tab (knappen rett over caps lock) for linjene i kodeblokken for å få innrykk. Funksjoner gjør programmeringen lettere.

Litt om funksjoner: Hva og hvorfor?

Funksjoner i Python (**Hva?**):

- ◇ består av en gruppe kodelinjer som utfører en spesifikk oppgave
- ◇ kan minne om matematiske funksjoner, men også Python-funksjoner
 - kan inneholde alle slags handlinger, ikke bare matematikk
 - kan returnere et svar (som `mat.funk.`), men kan også la være
 - kan ta argumenter, men kan også la være

Hvorfor bruke funksjoner?

- Dele et større program opp i mindre deler
 - lettere å lese, skrive og vedlikeholde kode
 - Lettere å forstå, enklere å teste, mer effektivt samarbeid
 - gir ryddigere og enklere kode
- Gjenbruk av kode
 - Vi kan bruke Pythons egne funksjoner, f.eks. `print()`, `round()`
 - Vi kan selv definere nye funksjoner og
 - * bruke dem flere steder i ett program,
 - * eller i flere ulike programmer

Her tar vi et eksempel på hvordan vi kan lage en funksjon som kan brukes flere ganger i programkoden. For eksempel kan vi skrive en funksjon som kan regne ut volumet av en sylinder med radius r og høyde h .

```
1 import numpy as np
2
3 def Volumsyylinder(r, h):
4
5     volum=np.pi*(r**2)*h
6     return volum
7
8 V=Volumsyylinder(3,5)
9 print(V)
```

I scriptet regner vi ut `Volumsyylinder(3,5)` volumet av sylindren med radius = 3 og høyde = 5. Funksjoner begynner med kontrollordet `def`, etterfulgt av navnet på funksjonen, parenteser og et kolon, der input-parameterne er gitt i parentes. Dersom du vil at funksjonen skal kunne ta ett eller flere argumenter, skriver du dem inn i parentesene. I dette eksemplet har vi deklartert variablene r (radius) og h (høyde), og det betyr at disse variablene inngår i funksjonen og de må defineres som et tall når funksjon skal regne ut volumet.

Legg merke til at etter funksjonshodet `def Volumsyylinder(r, h):` kommer resten av koden i linjene under med innrykk (bruke TAB-tasten). Så utføres utregningen av volumet, før vi til slutt har med `return volum` som skal returnere svaret funksjonen gir, og vi kan bruke videre i koden. Istedenfor `Volumsyylinder(3,5)`, kunne vi bruke:

```
1 a=3
2 b=5
3 V=Volumsyylinder(a,b)
4 print(V)
```

Ellers variablene kan leses inn i programmet fra skjermen:

```
1 r= float(input('Skriv inn radiusen til en sylinder: '))
2 h= float(input('Skriv inn høyden til en sylinder: '))
3 V=Volumsyylinder(r,h)
```

Eksempel 2.11.1: Her er 3 eksempler med funksjoner uten argument, med argumenter, og med matematiske beregninger og hvordan disse kan brukes (*kalles*).

```
1 def minfunk():
2     print("Her er en funksjon!")
3
4 def minfunkmedarg(Navn, Hilsen):
5     print("Hei %s, %s" %(Navn,Hilsen))
6
7 def minfunktall(x,y):
8     return x**2 + y**2
9
10 minfunk()
11
12 minfunkmedarg("Kari Olsen","Ha en fin dag!")
13
14 a = minfunktall(1,2)
15 print(a)
```

Ved å kjøre programkoden får vi:

```
1 Her er en funksjon!
2 Hei Kari Olsen, Ha en fin dag!
3 5
```

Eksempel 2.11.2: For eksempel setter følgende program opp funksjonen $f(x) = x^3 + 3x$ og lister opp på skjermen x og $f(x)$ under x og $f(x)$:

```
1 def f(x):
2     return x**3 + 3*x
3
4 print(f'x\t f(x)')
5
6 for x in range(10):
7     print(f'{x}\t {f(x)}')
```

Ved å kjøre programmet, ser resultatet slik ut:

```
1 x  f(x)
2 0  0
3 1  4
4 2  14
5 3  36
6 4  76
7 5  140
8 6  234
9 7  364
10 8  536
11 9  756
```

Eksempel 2.11.3: Her tas et eksempel med en funksjon som beregner skalarproduktet til to vektorer:

```
1 def skalarprod(a, b):
2     p= 0
3     for i in range(len(a)):
4         p += a[i]*b[i]
5     return p
6 # Prøv funksjonen:
7 a = [1, 2, 3]
8 b = [0, 3, -1]
9 ab = skalarprod(a,b)
10 print(f'Skalarproduktet av a og b er {ab:.2f}')
```

Ved å kjøre programmet, får man:

```
1 Skalarproduktet av a og b er 3.00
```

Oppg. 2.11.1: Hva får man ved å kjøre følgende program:

```
1 def skalarprod(m, n):
2     d= 0
3     for i in range(len(m)):
4         d += m[i]*n[i]
5     return d
6 # Prøv funksjonen:
7 m = [-1, 3, 0]
8 n=  [2, 1, 5]
9 mn = skalarprod(m,n)
10 print(f'Skalarproduktet av m og n er {mn:.1f}.')
```

Oppg. 2.11.2: a) Skriv et program som setter opp en funksjon for arealet til en sirkel med radius r og regn ut arealet til en sirkel med radius 5.

b) Skriv et program som leser inn radiusen til en sirkel fra brukeren og regner og skriver ut arealet til en sirkel.

Oppg. 2.11.3: Summene nedenfor beregner $\ln x$ når n velges stor nok.

$$s_1(x, n) = \sum_{i=1}^n (-1)^{i-1} \frac{1}{i} (x-1)^i, \quad 0 < x \leq 2$$

$$s_2(x, n) = \sum_{i=1}^n \frac{1}{i} \left(1 - \frac{1}{x}\right)^i, \quad x > \frac{1}{2}$$

Vi ønsker å lage en programkode som hjelper til å beregne disse summene for en gitt x og n . Vi vil studere estimeringsavviket (feilen) man får ved å bruke summen $s(x, n)$ for en gitt x , istedenfor $\ln x$. Estimeringsfeilen for s_1 kan studeres ved to forslag:

- 1) Å bestemme feilen ved å regne ut differansen mellom den eksakte verdien og den estimerte verdien: $\ln(1.5) - s(1.5, n)$ (naiv metode).
- 2) Å Regne ut det først leddet i summen s_1 som ikke er tatt med i beregningen av summen, som kan kalles det numeriske estimeringsavviket. To programkoder er skrevet slik:
For s_2 kan man se nærmere på 1).

```

1 # Programkode 1
2 import numpy as np
3
4 def S1(x, n):
5     S1 = 0
6     for i in range(1, n+1):
7         S1 += (-1)**(i-1) * (x-1)**i / i
8     return S1
9
10 def E1(x, i):
11     E1 = 0
12     E1 = (x-1)**i / i
13     return E1
14
15 x = 1.5
16 n=100
17 e=0.000001
18
19 for i in range(1, n+1):
20     EstimertLog1 = S1(x, i)
21     Estimeringsfeil1 = E1(x, i)
22     EksaktLog = np.log(x)
23     NaivEstimeringsfeil = abs(EstimertLog1 - EksaktLog)
24     if NaivEstimeringsfeil < e:
25         break
26
27 print(f' Ved å bruke {i} første ledd: Estimert= {EstimertLog1}, Eksaktverdi= {
    EksaktLog}, Naiv estimeringsfeil = {NaivEstimeringsfeil}, Estimeringsfeil = {
    Estimeringsfeil1}')

```

```

1 # programkode 2
2 import numpy as np
3
4 def S2(x, n):
5     S2 = 0
6     for i in range(1, n+1):
7         S2 += (1-1/x)**i / i
8     return S2
9
10 x = 1.5
11 n=100
12 e=0.000000001
13
14 for i in range(1, n+1):
15     EstimertLog2 = S2(x, i)
16     EksaktLog = np.log(x)

```

```

17     NaivEstimeringsfeil = abs(EstimertLog2 -EksaktLog )
18     if NaivEstimeringsfeil < e:
19         break
20 print(f'Ved å bruke {i} første ledd: Estimert= {EstimertLog2}, Eksaktverdi= {
    EksaktLog }, Naiv estimeringsfeil = {NaivEstimeringsfeil}')

```

Ved å kjøre programmeringskoder får man disse to visningene på skjermen (som ikke nødvendigvis er skrevet i samme rekkefølge som programkodene):

```

1 Ved å bruke 15 første ledd: Estimert= 0.4054657568451514, Eksaktverdi=
    0.4054651081081644, Naiv estimeringsfeil = 6.487369870189497e-07,
    Estimeringsfeil = 2.0345052083333333e-06

```

```

1 Ved å bruke 16 første ledd: Estimert= 0.40546510744296105, Eksaktverdi=
    0.4054651081081644, Naiv estimeringsfeil = 6.652033368581556e-10

```

a) Forklar hvilken programkode som kan brukes til å beregne $\ln(0.25)$, $\ln(1.75)$ og $\ln(2.25)$?
b) Hvordan kan man forbedre presisjonen og få bedre nøyaktighet?

Oppg. 2.11.4: Arealet til en trekant med sidelengdene a , b og c er gitt ved:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

der $s = \frac{1}{2}(a+b+c)$.

Skriv en programkode som beregner arealet til en trekant for en gitt a , b og c .

Oppg. 2.11.5: Kinetisk energi for en partikkel med masse m [kg] som har hastighet v [m/s] er gitt ved:

$$E = \frac{1}{2}mv^2$$

Skriv en programkode som definerer en funksjon for kinetisk energi, regner ut den kinetiske energien for en partikkel som har massen $m = 0.0001$ [kg] og hastigheten $v = 8$ [m/s].

Å kalle inn en funksjon i en annen funksjon

Eksempel 2.11.4: Her tas et eksempel med en funksjon som finner det største tallet blant tre tall ved å kalle en funksjon i en annen funksjon

```
1 def maksavto( x, y ):
2     if x > y:
3         return x
4     return y
5 def maksavtre( x, y, z ):
6     return maksavto( x, maksavto( y, z ) )
7 print(maksavtre(2, -3, 7))
```

Eksempel 2.11.5: Her tas et eksempel med en funksjon som kalles inn i en annen funksjon. Skriv en programkode for en funksjon $f1(x)$ som multipliserer argumentet x med 4 og en annen funksjon $f2(x)$ som legger til tallet 5 til argumentet x . Utvid programkoden til å regne ut $f1(f2(2))$ og $f2(f1(2))$. Ellers formuleres dette slik også:

Gitt $f(x) = 4x$ og $g(x) = x + 5$. Skriv disse som funksjoner i programkode, og bruk programmet til å regne ut $f(g(2))$ og $g(f(2))$.

```
1 def f1(x):
2     y= 4*x
3     return y
4
5 def f2(x):
6     y=x+5
7     return y
8
9 print(f1(f2(2)))
10 print(f2(f1(2)))
```

Svaret ved å kjøre programmet blir:

```
1 28
2 13
```

Sjekker: $f1(f2(2))$, der $f2(2) = 2 + 5 = 7$ og $f1(f2(2)) = f1(7) = 4 \cdot 7 = 28$.

Sjekker $f2(f1(2))$, der $f2(f1(2)) = f2(8) = 8 + 5 = 13$. ◇

Oppg. 2.11.6: Hva skrives ut som resultat av kjøring av følgende programkode?

```
1 def printut(a):
2     b = 3*a
3     print('b = %g' % b)
4 x= 5
5 print('x = %g' % x)
6 printut(x)
```

2.12 Plotting i Python

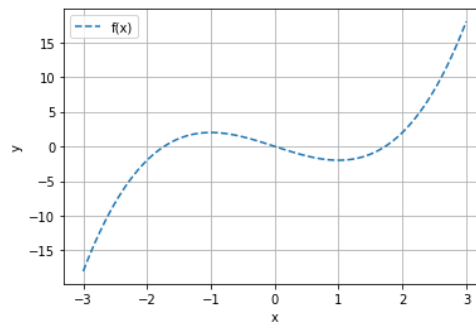
Vi vil ofte ha behov for å visualisere data. Da biblioteket *Matplotlib* hjelpe oss.

Plotting av kurver $y = f(x)$

Her skal vi bruke modulen pyplot (som er en del av biblioteket Matplotlib) for å tegne en graf:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-3,3, num = 101)
5 plt.yticks([-15,-5,5,15])
6 f = x**3-3*x
7
8 plt.plot(x,f, 'b--')
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'], loc='best')
12 plt.grid()
13 plt.show
14 plt.savefig('MinPythonFigMappe/eksplot.png')
```

Kjører vi dette scriptet, vil følgende figur plottes *og lagres*.



Elementene vi har tatt i bruk, her er følgende:

- `plt.plot(x,f, 'b--')` tegner grafen til funksjonen og tar inn x -verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'`, som skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med blåfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
- Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
- `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y -aksen.
- `plt.grid()` legger på et gitter bak grafen.
- `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
- `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best'`: plasserer tekstboksen der det passer best.
- `plt.show` tegner diagrammet i konsollvinduet.
- `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.

KONTROLL SPØRSMÅL. Hvordan kan du endre graftykkelsen og farger på grafen?
Hvordan kan du endre programkoden ved å skrive f ved hjelp av funksjonssyntaksen? ■

Oppg. 2.12.1: Følgende programkode er gitt:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-2,2, num = 101)
5 plt.yticks([-10,-5,5,10])
6 f = x**5-5*x**2
7
8 plt.plot(x,f, 'g--')
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'], loc='best')
12 plt.grid()
13 plt.show
14 plt.savefig('MinPythonFigMappe/EksGraf.png')
```

- a) Forklar hva programmet utfører linje for linje.
- b) Gjør de nødvendige endringene for at programkoden tegner grafen til: $g(x) = x^3 - 5x$ når $-2 \leq x \leq 2$.

Oppg. 2.12.2: La f være funksjonen gitt ved $f(x) = x^3 + 6x - 3$.

- a) Skriv en programkode som setter opp verditabell for x -verdiene mellom -2 og 2 (med å dele opp intervallet med steglengde = 0.01), og tegner grafen til f .
Utvid programmet til å tegne grafen til funksjonen
- b) Utvid programmet ditt til å la brukeren velge minste og største verdi for x og trinnene, i tabellen.

Oppg. 2.12.3: Skriv en programkode som tegner grafen til følgende funksjoner i samme koordinatsystem:

$$f(x) = \sin(2x - \pi)$$

$$g(x) = \cos(2x + \pi)$$

i intervallet $[-2\pi, 2\pi]$.

Skisser også $h(x) = \cos(2x + \frac{\pi}{2})$. Hva kan vi si om f og h ?

Utvid programmet for at den kan tegne grafen til funksjonene:

$$f(x) = c + a \sin\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

$$g(x) = c + a \cos\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

gitt $c = 60$, $a = 120$, $T = 365$, $k = 2$ og $d = 182.5$.

Plotting av flater $z = f(x, y)$

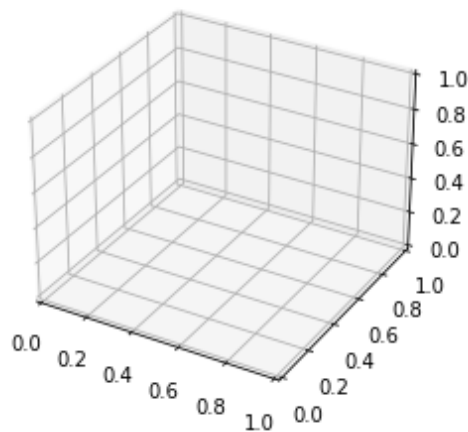
Størrelsen på gridet må tilpasses til hver situasjon. Her i dette eksemplet er brukt 101 punkter (100 oppdelinger) per akse, noe som gjør at flaten ser glatt ut.

To vanlige kommandoer for å plote flater i Python er `mesh` og `surf`. Forskjellen på dem er at `surf` fyller maskene mens `mesh` tegner kun maskenettet. Kommandoene brukes likt:

```
1 mesh(x,y,h)
2 surf(x,y,h)
```

Overflateplottet er en representasjon av tredimensjonale datasett. Den beskriver et funksjonelt forhold mellom to uavhengige variabler X og Y og en utpekt avhengig variabel Z . Det er et ledsagerplott av konturplottet. Dette bidrar til å skape topologien til overflaten som blir visualisert.

```
1 import matplotlib.pyplot as plt
2
3 fig = plt.figure()
4 ax = plt.axes(projection='3d')
```



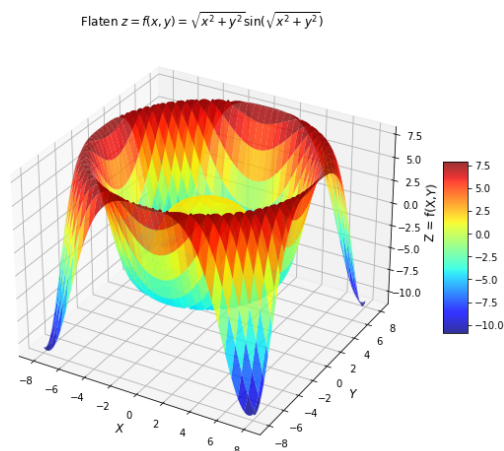
For å endre fargene som vises i `mesh` og `surf` skriv `cmap='jet'` eller istedenfor `jet`, kan man sette `hot`, `bone`, `cool`, osv.

For å se hvilke farger som representerer hvilke verdier kan man skrive `verb|colorbar()`. Til slutt kan vi pynte litt på figuren. For å angi navn på aksene bruker vi

```
1 ax.set_xlabel('$X$', fontsize=12)
2 ax.set_ylabel('$Y$', fontsize=12)
3 ax.set_zlabel('$Z = f(X,Y)$', fontsize=12)
4 # ax.set_title('Flaten $z=f(x,y)$');
5
6 # Justerer plasseringene til tittel og aksene
7 ax.set_title(" " * 35 + "Flaten $z=f(x,y)$" + " " * 35)
```

Eksempel 2.12.2: Skisser grafen til $z = \sqrt{x^2 + y^2} \sin(\sqrt{x^2 + y^2})$

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 n=101
5 # Setter opp definisjons området til de uavhengige x og y variablene:
6 X, Y = np.meshgrid(np.linspace(-8, 8, n), np.linspace(-8, 8, n))
7
8 # Setter opp funksjonen
9
10 def f(x, y):
11     t=np.sqrt(x ** 2 + y ** 2)
12     return t*np.sin(t)
13
14 Z= f(X,Y)
15
16 # Det fungerer også med:
17 # t = np.sqrt(X**2 + Y**2)
18 # Z = t*np.sin(t)
19
20 fig = plt.figure(figsize=(10,8))
21 ax = fig.add_subplot(111, projection = '3d')
22
23 # 3D flate
24 surf=ax.plot_surface(X, Y, Z, cmap='jet', alpha=0.8)
25
26 # colorbar
27 fig.colorbar(surf, shrink=0.4, aspect=7)
28
29 # Setter opp tekst på aksene og grafen
30
31 ax.set_xlabel('$X$', fontsize=12)
32 ax.set_ylabel('$Y$', fontsize=12)
33 ax.set_zlabel('$Z = f(X,Y)$', fontsize=12)
34 # ax.set_title('Flaten $z=f(x,y)$');
35
36 # Justerer plasseringene til titel og aksene
37 ax.set_title(" " * 35 + "Flaten $z=f(x,y)$" + " " * 35)
38
39 plt.show()
```



◇

2.13 Lister

Lister er mengden av elementer atskilt med komma mellom hakeparenteser, for eksempel `[1, 2, 3]` og `['Klara', 'Augusta', 7]`. Lister kan brukes til å lagre objekter underveis i programmet. For å lage en liste, oppretter man en tom liste ved å tilordne verdien `[]` til en variabel, for eksempel `liste=[]` eller `xverdier=[]`. Her er variabelnavnene «liste» og «xverdier».

```
liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']:
```

Kommando	Resultat	Forklaring
<code>liste[0]</code>	<code>'a'</code>	Første element er nr. 0
<code>liste[0:3]</code>	<code>['a', 'b', 'c']</code>	Tar ikke med nr. 3 (fjerde)
<code>liste[:3]</code>	<code>['a', 'b', 'c']</code>	Tar med element på plass nr. 0, 1 og 2
<code>liste[-1]</code>	<code>'h'</code>	Siste element er nr. -1
<code>liste[3:6]</code>	<code>['d', 'e', 'f']</code>	Fra og med nr. 3 til nr. 6
<code>liste[3:-1]</code>	<code>['d', 'e', 'f', 'g']</code>	Fra nr. 3, tar ikke med siste element
<code>liste[3:]</code>	<code>['d', 'e', 'f', 'g', 'h']</code>	Fra nr. 3, tar ikke med siste element

For å legge til elementer i en liste, brukes kommandoen «*append*», og listevariabelen brukes som forstavelse, atskilt med et punktum. I koden nedenfor opprettes det først en tom liste i linje 1, og i de tre neste linjene legger vi inn tallene 3 og 1 og bokstaven «c». I linje 5 skriver vi lista til skjerm.

```
1 liste = []
2 liste.append(3)# legger tallet 3 inn i lista
3 liste.append(1)# legger tallet 1 inn i lista
4 liste.append('c')# legger bokstaven c inn i lista
5 print(liste)
```

Når vi kjører programmet, får vi dette resultatet:

```
1 [3, 1, 'c']
```

Kontrollordet *len* kan gi oss lengden på lista, *len(liste)*, altså hvor mange elementer den inneholder:

```
1 liste=[1,7,2,4,3,3,5,6]# en liste med sju elementer
2 print(len(liste))
```

Svaret blir da 8.

Eksempel 2.13.1: Skriv en programkode som gir `[0, 1, 'e', 'p']`.

```
1 liste = []
2 liste.append(0)
3 liste.append(1)
4 liste.append('e')
5 liste.append('p')
6 print(liste)
```

Man kan også legge til flere elementer samtidig med programkoden

```
1 liste = []
2 liste.extend([0,1,'e','p'])
```

Oppg. 2.13.1: Hva blir resultatet av følgende?

a) Programkoden:

```
1 A=[1,2,3,4,5]
2 B=[2,4,6,8,10]
3 C=A+B
4 print(C)
```

b) Programkoden:

```
1 A=[1,2,3]
2 A+= [4]
3 A = A + [5]
4 print(A)
```

- c) En liste er gitt som: Liste=[1, 2, 3, 4, 5, 6] Bruk en for-løkke til å regne ut produktet av alle elementene i liste.

Oppg. 2.13.2: La f være funksjonen gitt ved $f(x) = -0.2x^2 + 1$. Skriv en programkode som setter opp x -verdiene, mellom -1 og 2 med steglengde 0.25 , og y -verdiene til f inn i to lister.

Oppg. 2.13.3: La f være funksjonen gitt ved $f(x) = -x^3 + x + 2$. Skriv en programkode som setter opp x -verdiene, mellom -2 og 3 med steglengde 0.25 , og y -verdiene til f inn i to lister.

2.14 Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
<code>f = open('filnavn')</code>	Åpner en fil, returnerer filreferanse
<code>f = open('filnavn','tilgang')</code>	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
<code>f.read()</code>	Returnerer hele innholdet av filen
<code>f.read(n)</code>	Returnerer <i>n</i> karakterer av innholdet
<code>f.readline()</code>	Returnerer neste linje (før / <i>n</i>)
<code>f.readlines()</code>	Returnerer hele innholdet av fila som en liste
<code>f.write(s)</code>	Skriver strengen <i>s</i> til fil
<code>f.writelines(liste)</code>	Skriver innholdet av liste av strenger til fil
<code>f.seek(offset fra hvor)</code>	Forflytter filpekeren (index) i filen
<code>f.tell()</code>	Returnerer posisjon til filpekeren i filen
<code>f.close()</code>	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

```
1 f = open('datafil.txt','r') # aapner fil for lesing
2 f = open('datafil.txt','w') # aapner fil for skriving
3 f = open(r'C:\User\files\fil.txt','a' )
4 # r først inni parentesen fo at \ ikke tolkes som spesielle tegn
5 # deretter leser filen fil.txt fra stien.
```

Eksempel 2.14.1:

```
1 f = open('fil1.txt', 'a')
2 f.write('Denne skrives i filen!')
3 print()
4 f.close()
5 print()
6 #åpen og les fra filen
7 f = open('fil1.txt', 'r')
8 print(f.read())
```

2.15 Laste inn, håndtere og plotte datasett

Vi har lært så langt hvordan vi bruke matematiske funksjoner til å regne funksjonsverdier og plotter disse. I naturvitenskapen vil ofte data / tall / målte verdier være hentet fra eksperimenter og observasjoner. Det kan ofte være store datasett med lange lister med tall. Vi skal nå hvordan vi kan laste inn data fra harddisken til vår pc eller eksterne servere og bruke innbygde Python-funksjoner og pakker for å håndtere slike lister.

pandas-biblioteket

pandas-biblioteket brukes til å analysere data.

```
1 import pandas as pd
2 df = pd.read_csv('data.csv')
```

I det neste eksempelet skal vi lese tid–temperatur-målinger, der vi bruker følgende syntakser for å åpne datafilen *tekopp-temp.csv* der i regnearket hver tekstlinje er en rad og verdiene for de ulike søylene er separert med komma (;). For å indikere dette, bruker man *sep=';'* og for å tolke desimaltall bruker man *decimal='.'*.

```
1 import pandas as pd
2 df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv', sep=';',
3                 decimal='.', comment='#', encoding='latin-1')
4 tid = df['Tid'].tolist()
5 temperatur = df['Temperatur'].tolist()
```

2.16 Modellering: tekopp-temperatur

Prosjekt: Modellering av tekopp-temperatur

Vi ønsker å studere endringen i tekopp temperatur i løpet av en time (60 min).

Læringsmål

- Utforske sammenhengen mellom konstant prosentvis endring, vekstfaktor og eksponentialfunksjoner
- Bruke funksjoner i modellering og argumentere for framgangsmåter og resultater,
- Modellere situasjoner knyttet til reelle datasett, presentere resultatene og argumentere for at modellene er gyldige
- Utforske matematiske egenskaper og sammenhenger ved å bruke programmering
- Tolk og bruke funksjoner i matematisk modellering og problemløsning.

Læringsutbytte

- Studentene skal forstå hvordan kurven som beskriver at temperaturfallet avtar raskest i starten for så å nærme seg temperaturen i rommet som er en asymptote til kurven.
- Studentene skal se at lineær kurvetilpasning ikke kan dekke behovet, spesielt i de første minuttene.

Programmeringsdidaktikk

Teori (det tas litt teori på tavla før eksperimentet begynner):

Newtons avkjølingslov: dette forsøket prøver å lage en matematisk modell som beskriver hvordan temperaturen endres i et objekt (en kopp med vann) som kjøles ned av omgivelsene. Funksjonsuttrykket:

$$y = f(t) = at + b$$

representerer en lineær funksjon som kan beskrive lineær sammenheng mellom to størrelser. Funksjonsuttrykket:

$$y = f(x) = c + ae^{bt}, \quad \text{der } b < 0$$

representerer en eksponentialfunksjon. Ved start ($t=0$) har funksjonen verdien $y = c + a$, og synker eksponentiell mot $y = c$ når tiden (t) går.

Utstyr:

- 10 eller flere kopper (samme type)
- Vannkoker
- En vekt
- En klokke
- Ett eller flere termometer(e)

Tidsbruk:

Forarbeid: 15-20 min og elevene jobber ca. 60 min, 10-15 min refleksjon over detaljene i prosjektet med åpen samtale og eventuelt kontrollspørsmål.

Gjennomføring:

Kok opp tilstrekkelig vann til å fylle opp alle koppene. Sett koppene på vekten etter tur og fyll opp med like mye av det varme vannet. Mål temperaturen med jevne mellomrom i løpet av en time (3600 sek), for eksempel etter 5 min (300 sek), 10 min (600 sek), ... og noter resultatet i tabellen.

Tid (sek)	Temp (Celsius)

For eksempel (data hentet fra time-temp.pdf (southampton.ac.uk))

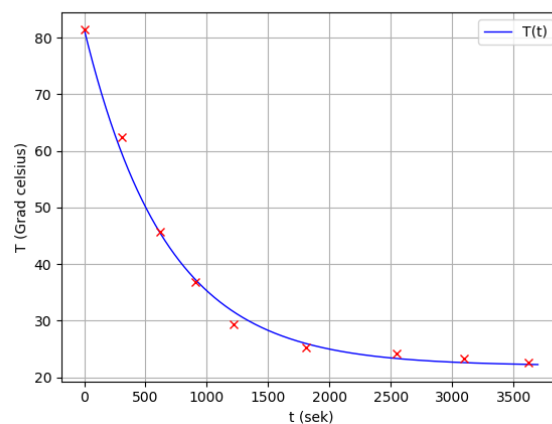
Tid (sek)	Temp (Celsius)
0	81.50
310	62.50
620	45.70
910	36.80
1220	29.40
1810	25.30
2550	24.20
3100	23.30
3620	22.50

Eksempel 2.16.1: I dette eksempelet skal vi nå se på filen ‘tekopp-temperatur.csv’. Hver linje i filen er en temperatur-måling i en tekopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 grader celsius. Her bruker vi Newtons avkjølingslov for å studere temperaturendringen i vannkoppen.

$$\frac{dT}{dt} = k(T - T_o)$$

der $T(t)$ ((målt i grader celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv', sep=';',
5                  decimal='.', comment='#', encoding='latin-1')
6
7 tid = df['Tid'].tolist()
8 temperatur = df['Temperatur'].tolist()
9 tstart = 0
10 Tstart = temperatur[0]
11 tslutt = 3700
12 deltat = 0.5
13 To = 22 # Rom temperatur
14 k = .0015 # proporsjonalitets koeff.
15 tverdier = []
16 Tverdier = []
17 t = tstart
18 T = Tstart
19 while t <= tslutt:
20     tverdier.append(t)
21     Tverdier.append(T)
22     t = t + deltat
23     deltaT = -deltat * k * (T - To)
24     T = T + deltaT
25
26 plt.plot(tverdier, Tverdier, linewidth=1, color='blue', label='T(t)')
27 plt.plot(tid, temperatur, 'x', color='red')
28 plt.grid()
29 plt.xlabel('t (sek)')
30 plt.ylabel('T (Grad celsius)')
31 plt.legend(loc='best')
```



2.17 Oppsummering

Matematiske operatører

Operator	Utfører	Eksempel	Resultat
+	Addisjon	$5 + 2$	7
-	Subtraksjon	$5 - 2$	3
*	Multiplikasjon	$5 * 2$	10
/	Divisjon	$5 / 2$	2.5
//	Heltalldivisjon	$13 // 3$	4
**	EkspONENT	$2 ** 3$	8
÷	Modulo	$13 \div 4$	1

Datatyper

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	{1, 3, 5}
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Å lese input fra brukeren og utskrift av tekst og variabel

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra tastaturet, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```
1 print('Her leses inn variablene')
2 a = int(input("Skriv inn a (et heltall) her: "))
3 b = float(input("Skriv inn b (et reelt tall) her: "))
4 alder=input("Skriv inn alderen din:")
5 print(f''Du er {alder} år.'')
```

Andre tegn og syntaks

Tegn	betydning
==	lik
!=	ikke lik
>	mer enn
>=	mer eller lik (minst)
<	mindre enn
<=	mindre eller lik (høyst)

Importerings av ulike pakker

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Man kan enten importere hele biblioteket eller det man trenger fra biblioteket.

```
1 from numpy import *
2 from numpy import pi sin cos
```

Noen kommandoer

<code>print ()</code>	Skriv på skjerm
<code>range(1,n+1)</code>	Alle hele tall fra og med 1 til n
<code>range(1,n+1,m)</code>	Alle hele tall fra og med 1 til n med m steg lengde
<code>range(1,n+1,m)</code>	Alle hele tall fra og med 1 til n med m steg lengde
<code>if ... elif ... else</code>	Vilkår, if-test.
<code>while</code>	Løkke som kjører til testen er usann.
<code>for</code>	Løkke som kjører et på forhånd kjent antall ganger.
<code>import matplotlib.pyplot as plt</code>	Importerer selektivt pyplot-modulen til matplotlib som <i>plt</i>
<code>plt.plot()</code>	Tegner grafen til to lister.

Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler.

```
1 def funksjonsnavn(variabler):
2     kodeblokk
```

- Første linje kalles *funksjonshode*, som består av `def`, fulgt av navnet på funksjonen i parenteser og variabler, som inngår i funksjonen et kolon :
- Resten av koden kalles kodeblokk, som hører til funksjonen som må tas med.

Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
<code>f = open('filnavn')</code>	Åpner en fil, returnerer filreferanse
<code>f = open('filnavn','tilgang')</code>	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
<code>f.read()</code>	Returnerer hele innholdet av filen
<code>f.read(n)</code>	Returnerer n karakterer av innholdet
<code>f.readline()</code>	Returnerer neste linje (før / n)
<code>f.readlines()</code>	Returnerer hele innholdet av fila som en liste
<code>f.write(s)</code>	Skriver strengen s til fil
<code>f.writelines(liste)</code>	Skriver innholdet av liste av strenger til fil
<code>f.seek(offset fra hvor)</code>	Forflytter filpekeren (index) i filen
<code>f.tell()</code>	Returnerer posisjon til filpekeren i filen
<code>f.close()</code>	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

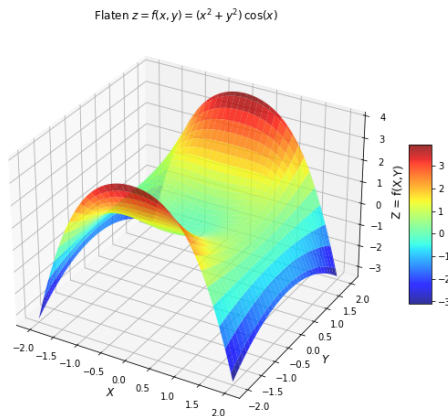
Fasit

Fasit 1.1.1:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 n=101
5 # Setter opp definisjons området til de uavhengige x og y variablene:
6 X, Y = np.meshgrid(np.linspace(-2, 2, n), np.linspace(-2, 2, n))
7
8 # Setter opp funksjonen
9
10 def f(x, y):
11     z=(x**2 + y**2)*np.cos(x)
12     return z
13
14 Z= f(X,Y)
15
16 # Det fungerer også med:
17 # Z = (X**2 + Y**2)*np.cos(X)
18
19 fig = plt.figure(figsize=(10,8))
20 ax = fig.add_subplot(111, projection = '3d')
21
22 # 3D flate
23 surf=ax.plot_surface(X, Y, Z, cmap='jet', alpha=0.8)
24
25 # colorbar
26 fig.colorbar(surf, shrink=0.4, aspect=7)
27
28 # Setter opp tekst på aksene og grafen
29
30 ax.set_xlabel('$X$', fontsize=12)
31 ax.set_ylabel('$Y$', fontsize=12)
32 ax.set_zlabel('$Z = f(X,Y)$', fontsize=12)
33 # ax.set_title('Flaten $z=f(x,y)$');
34
35 # justerer plasseringene til titel og aksene
36 ax.set_title(" " * 35 + "Flaten $z=f(x,y)=(x^2+y^2) \cos(x)$" + " " * 35)
37
38 plt.show()
```

Fasit 1.3.1:

- Programmet beregner numerisk differensialligningen $y' = f(x, y) = 2y$ gitt $y(0) = 1$ i intervallet $[0, 0.5]$. Det gjøres med 2 steg (steglengde $h = 0.25$) for å bestemme $y(0.5)$.
- Programmet bruker Eulers metode.
 - Først defineres funksjonen $f(x, y) = 2y$.
 - Startbetingelsen er $x_{\text{start}} = 0$ og $y(0) = 1$, og sluttpunktet er $x_{\text{slutt}} = 0.5$.



- Med 2 steg blir steglengden $h = \frac{0.5-0}{2} = 0.25$.
- Det lages en array x med punktene $x = 0, 0.25, 0.5$.
- Eulers metode: $y_{n+1} = y_n + h \cdot f(x_n, y_n)$ brukes i en løkke for å beregne $y(0.25)$ og $y(0.5)$.
- Til slutt skrives resultatene ut og sammenlignes med den eksakte løsningen $y(x) = e^{2x}$.

c) i) Med 2 steg ($h = 0.25$):

$$y_0 = y(0) = 1$$

$$\text{Steg 1: } y(0.25) = y_1 = y_0 + h \cdot f(0, y_0) = 1 + 0.25 \cdot (2 \cdot 1) = 1 + 0.5 = 1.5$$

$$\text{Steg 2: } y(0.5) = y_2 = y_1 + h \cdot f(0.25, y_1) = 1.5 + 0.25 \cdot (2 \cdot 1.5) = 1.5 + 0.25 \cdot 3 = 1.5 + 0.75 = 2.25$$

Dette stemmer med programmets utskrift: $y[1] = 1.5000$ og $y[2] = 2.2500$.

ii) Med 5 steg ($h = 0.1$):

n	x_n	y_n	$y_{n+1} = y_n + 0.1 \cdot (2y_n)$
0	0.0	1.0000	$1.0000 + 0.2 = 1.2000$
1	0.1	1.2000	$1.2000 + 0.24 = 1.4400$
2	0.2	1.4400	$1.4400 + 0.288 = 1.7280$
3	0.3	1.7280	$1.7280 + 0.3456 = 2.0736$
4	0.4	2.0736	$2.0736 + 0.41472 = 2.48832$
5	0.5	2.48832	

Den numeriske løsningen med 5 steg blir $y(0.5) \approx 2.48832$.

d) Den eksakte løsningen er $y(0.5) = e^{2 \cdot 0.5} = e^1 \approx 2.7182818$.

- Med 2 steg ($h = 0.25$) får vi $y(0.5) = 2.25$. Feilen er $|2.71828 - 2.25| = 0.46828$.
- Med 5 steg ($h = 0.1$) får vi $y(0.5) \approx 2.48832$. Feilen er $|2.71828 - 2.48832| = 0.22996$.

Vi ser at feilen blir mindre når vi øker antall steg (reduserer steglengden h). Dette viser at Eulers metode konvergerer mot den eksakte løsningen når $h \rightarrow 0$.

Fasit 1.4.1:

```

a) 1 import numpy as np
   2 import matplotlib.pyplot as plt
   3
   4 def f(t,y,z):
   5     return 2*y - z + 3
   6 def g(t,y,z):
   7     return y + 2*z
   8
   9 tstart = 0
  10 tslutt = 1
  11

```

```

12 h = 0.001
13 N = int((tslutt - tstart)/h)
14
15 t = np.zeros(N+1)
16 for i in range(N+1):
17     t[i] = tstart + i*h
18
19 # Initialverdier
20 y = np.zeros(N+1)
21 z = np.zeros(N+1)
22 y[0] = -1
23 z[0] = 1
24
25 # Eulers metode for system
26 for i in range(N):
27     y[i+1] = y[i] + f(t[i], y[i], z[i]) * h
28     z[i+1] = z[i] + g(t[i], y[i], z[i]) * h
29
30 plt.grid(ls='--')
31 plt.plot(t, y, label='y')
32 plt.plot(t, z, label='z')
33 plt.legend()
34
35 print("=" * 45)
36 print(f"\nDen numeriske løsningen gir:")
37 print(f"y({tslutt}) = {y[N]:.6f}")
38 print(f"z({tslutt}) = {z[N]:.6f}")
39 print(f"Antall steg brukt: {N}")
40 print(f"Steglengde h: {h:.4f}")

```

Med å kjøre programmet får man:

```

1 =====
2
3 Den numeriske løsningen gir:
4 y(1) = -2.880410
5 z(1) = 3.439570
6 Antall steg brukt: 1000
7 Steglengde h: 0.0010

```

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(t,y,z,w):
5     return 2*y - z + w + 3
6 def g(t,y,z,w):
7     return y + 2*z - w
8 def h(t,y,z,w):
9     return 3*y - 2*z + 2*w
10
11 tstart = 0
12 tslutt = 1
13
14 h = 0.001
15 N = int((tslutt - tstart)/h)
16
17 t = np.zeros(N+1)
18 for i in range(N+1):
19     t[i] = tstart + i*h
20
21 # Initialverdier

```

```

22 y = np.zeros(N+1)
23 z = np.zeros(N+1)
24 w = np.zeros(N+1)
25 y[0] = -1
26 z[0] = 1
27 w[0] = 2
28
29 # Eulers metode for system
30 for i in range(N):
31     y[i+1] = y[i] + f(t[i], y[i], z[i], w[i]) * h
32     z[i+1] = z[i] + g(t[i], y[i], z[i], w[i]) * h
33     w[i+1] = w[i] + h(t[i], y[i], z[i], w[i]) * h
34
35 plt.grid(ls='--')
36 plt.plot(t, y, label='y')
37 plt.plot(t, z, label='z')
38 plt.plot(t, w, label='w')
39 plt.legend()
40
41 print("=" * 45)
42 print(f"\nDen numeriske løsningen gir:")
43 print(f"y({tslutt}) = {y[N]:.6f}")
44 print(f"z({tslutt}) = {z[N]:.6f}")
45 print(f"w({tslutt}) = {w[N]:.6f}")
46 print(f"Antall steg brukt: {N}")
47 print(f"Steglengde h: {h:.4f}")

```

Med å kjøre programmet får man:

```

1 =====
2
3 Den numeriske løsningen gir:
4 y(1) = 10.862872
5 z(1) = -1.040795
6 w(1) = 17.594097
7 Antall steg brukt: 1000
8 Steglengde h: 0.0010

```

b) Fasit 1.5.1:

```

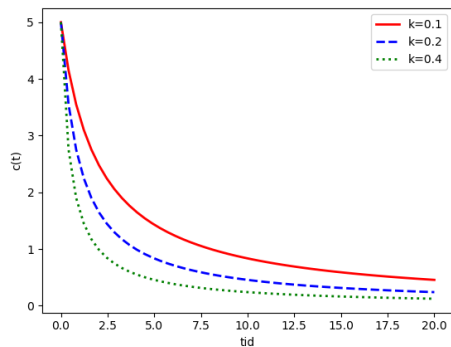
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # funksjonen setter opp dx/dt
6 def model(c,t,k):
7     dcdt = -k * c**2
8     return dcdt
9
10 # startverdi
11 c0 = 5
12 # tidslinjen
13 t = np.linspace(0,20)
14
15 # Løser ODEs
16 k = 0.1
17 c1 = odeint(model,c0,t,args=(k,))
18 k = 0.2
19 c2 = odeint(model,c0,t,args=(k,))
20 k = 0.4
21 c3 = odeint(model,c0,t,args=(k,))

```

```

22
23 # Tegner løsningskurvene
24 plt.plot(t,c1,'r-',linewidth=2,label='k=0.1')
25 plt.plot(t,c2,'b--',linewidth=2,label='k=0.2')
26 plt.plot(t,c3,'g:',linewidth=2,label='k=0.4')
27
28 plt.xlabel('tid')
29 plt.ylabel('c(t)')
30 plt.legend()
31 plt.show()

```

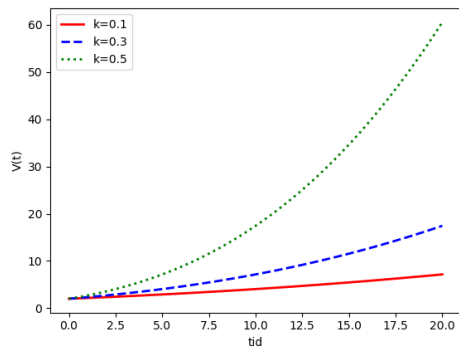


Fasit 1.5.2:

```

1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # funksjonen setter opp dx/dt
6 def model(V,t,k):
7     dVdt = k * V**(2/3)
8     return dVdt
9
10 # startverdi
11 V0 = 2
12 # tidslinjen
13 t = np.linspace(0,20)
14
15 # Løser ODEs
16 k = 0.1
17 V1 = odeint(model,V0,t,args=(k,))
18 k = 0.3
19 V2 = odeint(model,V0,t,args=(k,))
20 k = 0.5
21 V3 = odeint(model,V0,t,args=(k,))
22
23 # Tegner løsningskurvene
24 plt.plot(t,V1,'r-',linewidth=2,label='k=0.1')
25 plt.plot(t,V2,'b--',linewidth=2,label='k=0.3')
26 plt.plot(t,V3,'g:',linewidth=2,label='k=0.5')
27 plt.xlabel('tid')
28 plt.ylabel('V(t)')
29 plt.legend()
30 plt.show()

```



Fasit 1.5.3:

- a) $x_0 = 0$ og $y(0,2) = y_1 = y_0 + hy'_0 = 2 + 0,2(0) = 2$
 $x_1 = 0,2$, $y_2 = y(0,4) = y_1 + hy'_1 = 2 + 0,2(3 \cdot 0,2e^{-2} = 2,01640$
 $x_2 = 0,4$, $y_3 = y(0,6) = y_2 + hy'_2 = 2,016 + 0,2(3 \cdot 0,2e^{-2,016} = 2,0482$
- b) $\int e^y dy = \int 3t dt$ gir $e^y = 3/2t^2 + C$ eller $y = \ln(3/2t^2 + C)$ med $y(0) = 2$ og dermed $C = e^2$ og $y(0,6) = \ln(3/2(0,6)^2 + e^2) = 2,075$ Svarene numerisk og eksakt er 2,0482 og 2,075, og er nære hverandre. Hvis man velger mindre h -verdi får en bedre presisjon.

```

c) 1 import numpy as np
    2
    3 def f(x,y):
    4     return 3*x*np.exp(-y)
    5
    6 xstart = 0
    7 xslutt = 0.6
    8
    9 h = 0.2
   10 N = int((xslutt - xstart)/h)
   11
   12 x = np.zeros(N+1)
   13 for i in range(N+1):
   14     x[i] = xstart + i*h
   15
   16 y = np.zeros(N+1)
   17 y[0] = 2
   18
   19 print("=" * 55)
   20 print("  n      x[n]      y[n]      dy/dx      y[n+1]")
   21 print("=" * 55)
   22
   23 for i in range(N):
   24     stigning = f(x[i], y[i])
   25     y[i+1] = y[i] + stigning * h
   26     print(f"n={i}  {x[i]:7.4f}  {y[i]:7.4f}  {stigning:7.4f}  {y[i+1]:7.4f}")
   27
   28 print("=" * 55)
   29 print(f"\nDen numeriske løsningen gir at y({xslutt}) = {y[N]:.6f}")
   30 print(f"Antall steg brukt: {N}")
   31 print(f"Steglengde h: {h:.4f}")

```

Ved å kjøre programmet, får vi:

```

1 =====
2  n      x[n]      y[n]      dy/dx      y[n+1]

```

```

3 =====
4 n=0      0.0000      2.0000      0.0000      2.0000
5 n=1      0.2000      2.0000      0.0812      2.0162
6 n=2      0.4000      2.0162      0.1598      2.0482
7 =====
8
9 Den numeriske løsningen gir at  $y(0.6) = 2.048240$ 
10 Antall steg brukt: 3
11 Steglengde h: 0.2000

```

Fasit 1.5.4:

- Programmet beregner numerisk differensialligningen $y' = f(x, y) = 0.5y$ gitt $y(0) = 1$ i intervallet $[0, 10]$ ved å dele opp intervallet i 4 like deler ($h = 2.5$)
- Programmet beregner numerisk differensialligningen $y' = f(x, y) = 0.5y$ gitt $y(0) = 1$ ved hjelp av Eulers metode i intervallet $[0, 10]$ med steglengde $h = 2.5$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Fra oppgaven
5 def f(x,y):
6     return 0.5*y
7
8 xstart = 0
9 xslutt = 10
10
11 # Definerer steglengde og antall steg
12 h = 2.5
13 N = 4
14
15 # Deler inn x-aksen
16 x = np.zeros(N+1)
17 for i in range(N+1):
18     x[i] = xstart + i*h
19
20 # Eulers metode
21 y = np.zeros(N+1)
22
23 # Initialbetingelsen
24 y[0] = 1
25
26 print('\tx[n]\ty[n]\t dy/dx \ty[n+1]')
27 # Løkke
28 for i in range(N):
29     y[i+1] = y[i] + f(x[i], y[i]) * h
30     print(f"n={i}   x[{i}]= {x[i]:7.4f}   y[{i}]= {y[i]:7.4f}   y[{i+1}]= {y[i+1]:7.4f}")
31
32 plt.plot(x,y, '--', label='Numerisk løsning')
33 plt.xlabel('x')
34 plt.ylabel('y')
35
36 """ Vi kan sjekke opp mot Den eksakte løsningen """
37 def yeksakt(x):
38     return np.exp(0.5*x)
39
40 xpen = np.linspace(xstart, xslutt, 100)
41 plt.plot(xpen, yeksakt(xpen), label='Den eksakte løsningen')
42 plt.legend()
43

```

```

44 print("=" * 45)
45 print(f"\nDen numeriske løsningen gir at y({xslutt}) = {y[N]:.6f}")
46 print(f"Antall steg brukt: {N}")
47 print(f"Steglengde h: {h:.4f}")
48 print(f"Den eksakte løsningen er at y({xslutt}) = {yeksakt(x[N]):.6f}")

```

- c) i) Den numeriske løsningen gir $y(10) = 25.628906$
 ii) $y(2.5) = y(0) + f(0, 1) \cdot 2.5 = 1 + 0.5 \cdot 2.5 = 1 + 1.25 = 2.25$
 $y(5) = y(2.5) + f(2.5, 2.25) \cdot 2.5 = 2.25 + 0.5 \cdot 2.25 \cdot 2.5 = 2.25 + 2.8125 = 5.0625$
 Dette stemmer med utskriften hvor $y[1] = 2.25$ og $y[2] = 5.0625$.
- d) Man kan redusere steglengden h (og dermed øke antall steg N) for å få bedre presisjon. For eksempel kan man sette $h = 0.1$ og $N = 100$.

Fasit 1.5.5:

- a) Programmet løser startverdiproblemet:

$$\frac{dy}{dt} = 0.02y, \quad y(0) = 2$$

i intervallet $t \in [0, 0.6]$.

- b) Programmet bruker **Eulers metode**. Prinsippet er å finne neste punkt ved å gå i retningen til den deriverte:

$$y_{n+1} = y_n + f(t_n, y_n) \cdot h$$

hvor h er steglengden. Dette tilsvarer å approksimere løsningskurven med en rett linje i hvert steg.

- c) **Forklaring av programmet:**

- **def f(t, y):** Definerer funksjonen som representerer høyre siden av differensialligningen.
- **t0 = 0, y0 = 2, T = 0.6:** Setter startbetingelser og sluttidspunkt.
- **hVerdier = [0.4, 0.2, 0.1]:** Liste over steglengder som skal testes.
- **for h in hVerdier:** Løkke som kjører Eulers metode for hver steglengde.
- Inne i løkken:
 - Regner ut antall steg N basert på steglengde h .
 - Oppretter arrays for t og y med lengde $N+1$.
 - Setter startverdier: $t[0]=0$, $y[0]=2$.
 - Skriver ut tabellhode for denne steglengden.
 - For-løkke som utfører N iterasjoner av Eulers metode og skriver ut hvert steg.
 - Beregner eksakt verdi og feil, og skriver ut resultatene.
- Til slutt lages et plott som sammenligner de numeriske løsningene med den eksakte løsningen.

- d) **a) Utregning for hånd med $h = 0.2$:**

$$f(t, y) = 0.02y, \quad y(0) = 2$$

$$n = 0: \quad t_0 = 0, \quad y_0 = 2$$

$$y(0.2) \approx y_1 = y_0 + f(0, 2) \cdot 0.2 = 2 + (0.02 \cdot 2) \cdot 0.2 = 2 + 0.04 \cdot 0.2 = 2 + 0.008 = 2.008$$

$$n = 1: \quad t_1 = 0.2, \quad y_1 = 2.008$$

$$y(0.4) \approx y_2 = y_1 + f(0.2, 2.008) \cdot 0.2 = 2.008 + (0.02 \cdot 2.008) \cdot 0.2$$

$$= 2.008 + (0.04016) \cdot 0.2 = 2.008 + 0.008032 = 2.016032$$

$$n = 2: \quad t_2 = 0.4, \quad y_2 = 2.016032$$

$$y(0.6) \approx y_3 = y_2 + f(0.4, 2.016032) \cdot 0.2 = 2.016032 + (0.02 \cdot 2.016032) \cdot 0.2$$

$$= 2.016032 + (0.04032064) \cdot 0.2 = 2.016032 + 0.008064128 = 2.024096128$$

Programmet vil for $h = 0.2$ skrive ut:

```

1 n=0    t[0]= 0.0000    y[0]=2.0000    0.0400    y[1]=2.0080
2 n=1    t[1]= 0.2000    y[1]=2.0080    0.0402    y[2]=2.0160
3 n=2    t[2]= 0.4000    y[2]=2.0160    0.0403    y[3]=2.0241

```

som stemmer overens med utregningen (avrundet til 4 desimaler).

b) Utskriften viser at når steglengden h halveres (fra 0.4 til 0.2 til 0.1), blir feilen i estimatet for $y(0.6)$ mindre:

- For $h = 0.4$: Feil ca. 3.28×10^{-4}
- For $h = 0.2$: Feil ca. 8.10×10^{-5}
- For $h = 0.1$: Feil ca. 2.04×10^{-5}

Dette er forventet, siden Eulers metode har en global feil som er proporsjonal med h .

e) **i) Endring i programmet:** Endre linjen $T = 0.6$ til $T = 1.2$. Utskriften for $T = 1.2$ vil bli omtrent:

```

1 --- Steglengde h = 0.4 (N = 3 steg) ---
2 Numerisk losning: y(1.2) = 2.04672000
3 Eksakt losning:   y(1.2) = 2.04840525
4 Feil: 1.69e-03
5
6 --- Steglengde h = 0.2 (N = 6 steg) ---
7 Numerisk losning: y(1.2) = 2.04808171
8 Eksakt losning:   y(1.2) = 2.04840525
9 Feil: 3.24e-04
10
11 --- Steglengde h = 0.1 (N = 12 steg) ---
12 Numerisk losning: y(1.2) = 2.04832142
13 Eksakt losning:   y(1.2) = 2.04840525
14 Feil: 8.38e-05

```

ii) Feilen er større for $y(1.2)$ enn for $y(0.6)$ fordi Eulers metode akkumulerer feil for hvert steg. For å nå $t = 1.2$ med samme steglengde h , må vi ta dobbelt så mange steg som for å nå $t = 0.6$. Hvert steg introduserer en liten lokal feil, og disse forplanter seg og akkumuleres. Dermed vil den globale feilen vokse med avstanden fra startpunktet, gitt en konstant steglengde.

Fasit 1.5.6:

c) Løsningen til $\frac{dy}{dt} = -ay(y - \frac{b}{a})$ er $y = \frac{b/a}{1 + Ce^{-bt}}$ Python program:

```

1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # funksjonen for dy/dt
6 def model(y,t,a,b):
7     dydt = -a * y*y + b*y
8     return dydt
9
10 # startverdi
11 y0 = 5
12
13 # deler opp tidsaksen
14 t = np.linspace(0,20)
15
16 # solve ODEs
17 a= 0.1
18 b=0.2
19 y1 = odeint(model,y0,t,args=(a,b,))
20 a= 0.1
21 b=0.01
22 y2 = odeint(model,y0,t,args=(a,b,))
23 a = 0.01

```

```

24 b=0.2
25 y3 = odeint(model,y0,t,args=(a,b,))
26
27 # plot results
28 plt.plot(t,y1,'r-',linewidth=2,label='a=0.1, b=0.2')
29 plt.plot(t,y2,'b--',linewidth=2,label='a=0.1, b=0.01')
30 plt.plot(t,y3,'g:',linewidth=2,label='a=0.01, b=0.2')
31 plt.xlabel('tid')
32 plt.ylabel('y(t)')
33 plt.legend()
34 plt.show()

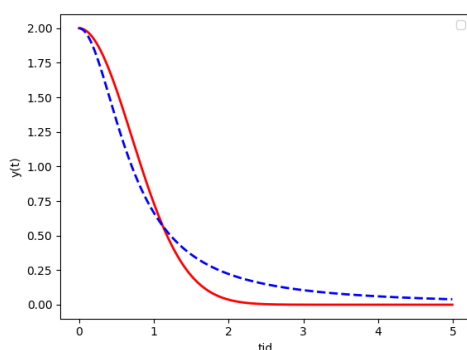
```

Fasit 1.5.7:

```

1 from scipy.integrate import odeint
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 def f(y, t):
6     return -2*y*t
7
8 def g(y, t):
9     return -2 * t * y**2
10
11 y0 = 2          # startverdi
12 a = 0           # integrasjonsgrense
13 b = 5
14
15 t = np.arange(a, b, 0.01) # tidslinjen for løsningen
16
17 y1 = odeint(f, y0, t) # Løser f gitt y0
18 y2 = odeint(g, y0, t) # Løser g gitt y0
19
20 plt.plot(t,y1,'r-',linewidth=2, label='dy/dt = -2ty')
21 plt.plot(t,y2,'b--',linewidth=2, label='dy/dt = -2ty^2')
22 plt.xlabel('tid')
23 plt.ylabel('y(t)')
24 plt.legend()
25 plt.show()

```



Fasit 1.5.8:

```

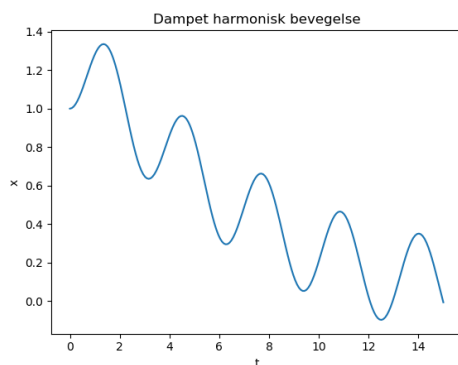
1 # Vi ønsker å løse x''+0.4x'+0.05x= cos(2t) gitt x(0)=1 og x'(0)=0
2 # Vi omskriver 2. orden differensial ligningen til et lineært system
3 # x' = y og y' = -0.4y - 0.05x + cos(2t)
4
5 import numpy as np

```

```

6 import matplotlib.pyplot as plt
7
8 def f(t, x, y):
9     return y
10
11 def g(t, x, y):
12     return -0.4*y - 0.05*x + np.cos(2*t)
13
14 tstart = 0
15 tslutt = 15
16
17 h = 0.01
18 N = int((tslutt - tstart)/h)
19
20 t = np.zeros(N+1)
21 for i in range(N+1):
22     t[i] = tstart + i*h
23
24 x = np.zeros(N+1)
25 y = np.zeros(N+1)
26 x[0] = 1
27 y[0] = 0
28
29 for i in range(N):
30     x[i+1] = x[i] + f(t[i], x[i], y[i]) * h
31     y[i+1] = y[i] + g(t[i], x[i], y[i]) * h
32
33 plt.xlabel("t")
34 plt.ylabel("x")
35 plt.title("Dempet harmonisk bevegelse")
36 plt.plot(t, x)
37 plt.grid(True)
38 plt.show()
39
40 print("=" * 45)
41 print(f"\nDen numeriske løsningen gir x({tslutt}) = {x[N]:.6f}")
42 print(f"Antall steg brukt: {N}")
43 print(f"Steglengde h: {h:.4f}")

```



Fasit 2.4.1:

a) `print('Denne setningen skrives ut!')`

b) P i print skal være liten bokstav. Mangler apostrof i enden til setningen.

Fasit 2.4.2:

a) 5+7

- b) 12
- c) SyntaxError. Legg merke til at det mangler en apostrof etter 7.
- d) Print skal være print.
- e)

```
1 5+7
```

Fasit 2.4.3:

- a)

```
1 > 3*5+4/5
2 15.8
```
- b)

```
1 > 2**3+3**2+1.25/4
2 17.25
```

- c) Resultatet til $18\%4$ blir 2 (restleddet i divisjonen 18 på 4).

Fasit 2.4.4:

- a) Skriver og kjører i console:

```
1 > 5*8/2
2 20.0
```

- b) Skriver og kjører i konsollvinduet:

```
1 > 7*10
2 70.0
```

- c) Skriver og kjører i konsollvinduet:

```
1 > 6*(8+10)/2
2 54.0
```

Fasit 2.6.1:

- a)

```
1 m=7
2 n=1
3 m=m+3 # eller m+=3
4 n=n*5 # eller n*=5
5 p= m*n
6 print('Produktet blir da',p)
```

Svaret blir da 30.

- b)

```
1 a=5
2 b='TALL'
3 a=a*2
4 b=b* 3
5
6 print(a)
7 print(b)
```

Utskriften er da:

```
1 10
2 TALLTALLTALL
```

Fasit 2.6.2:

```
1 l = 5
2 b = 4
3 h = 3
4 volum=l*b*h
5 overflate = 2*l*b + 2*b*h + 2*l*h
6 print(f"Volume er {volum}.")
7 print(f"Overflaten er {overflate}.")
```

Ved å kjøre programmet blir svaret:

```
1 Volume er 60.  
2 Overflaten er 94.
```

Fasit 2.6.3:

```
1 h= 3.5  
2 g = 9.81  
3 v0 = 20  
4 t = 5  
5  
6 y= h - 0.5*g*t**2+v0*t  
7  
8 print('y er lik',y)
```

Fasit 2.7.1:

```
1 a = float(input("Skriv inn a (et reelt tall) her: "))  
2 b= 3  
3 c= 4  
4 d = (1/3)*a*b+c**2  
5  
6 print('d er lik',d)
```

Fasit 2.7.2:

```
1 r = float(input("Skriv inn radiusen til en sylinder (et reelt positivt tall)  
    her: "))  
2 h= float(input("Skriv inn radiusen til en sylinder (et reelt positivt tall) her  
    : "))  
3 pi=3.14  
4 V = pi*r**2*h  
5  
6 print('Volumet til sylindere er lik',V)
```

Fasit 2.8.1:

```
1 import numpy as np  
2  
3 x=0.5  
4 y = np.sin(np.pi*x)  
5  
6 print('y er lik',y)
```

Fasit 2.8.2:

a) Programkode:

```
1 import numpy as np  
2 svar1=np.cos(2)+np.tan(1.5)  
3 print(svar1)
```

b) Programkode:

```
1 import numpy as np  
2 svar2=np.exp(4)+np.log10(2)+np.log(0.1)  
3 print(svar2)
```

Fasit 2.8.3:

```

1 import pylab
2
3 x=0.5
4 y = sin(pi*x)
5
6 print('y er lik',y)

```

Fasit 2.10.1:

Hvis man kjører programmet, blir man spurt **Skriv et tall**, og når man skriver hvis tallet er negativt, blir den erstattet med det tallet med positiv fortegn. Til slutt skriver programmet ut **Absluttverdien til a er x**, der x er et positivt tall uansett hva input a var. Resultatet blir Absluttverdien til -7 er 7.

Fasit 2.10.2:

a) Programkode:

```

1 tall = int(input("Skriv inn et tall: "))
2 if tall > 0:
3     print(f"Tallet {tall} er positivt.")
4 elif tall < 0:
5     print(f"Tallet {tall} er negativt.")
6 else:
7     print(f"Tallet {tall} er null.")

```

b) Programkode:

```

1 tall = int(input("Skriv inn et positivt tall: "))
2
3 if tall == 0 or tall < 0:
4     print(f"{tall} er ikke positivt, skriv et positivt tall.")
5 elif tall%2 == 0:
6     print(f"Tallet {tall} er jevntall.")
7 else:
8     print(f"Tallet {tall} er oddetall.")

```

c) Programkode:

```

1 Tall1 = float(input("Skriv inn et tall: "))
2 Tall2 = float(input("Skriv inn et tall: "))
3
4 if Tall1*Tall2 < 0:
5     print(f"Tallene {Tall1} og {Tall2} har motsatt fortegn.")
6 elif Tall1*Tall2 > 0:
7     print(f"Tallene {Tall1} og {Tall2} har samme fortegn.")
8 else:
9     print(f"Et av tallene {Tall1} eller {Tall2} er null.")

```

Fasit 2.10.3:

Programkoden spør hva summen $49 + 35$, og når brukeren svarer, programkoden svarer om svaret ikke er rett (*Regn igjen!*) eller om det er rett (*Du klarte det!*).

Fasit 2.10.4:

```

1 innlest = input("Hva er 52+19?")
2 tall = int(innlest)
3
4 if tall != 71:
5     while tall != 71:
6         innlest = input("Feil. Regn igjen!")
7         tall = int(innlest)
8     else:
9         print("Du klarte det!")

```

Fasit 2.10.5:

```
1 tall = int(input("Skriv inn et oddetall: "))
2
3 if tall%2 != 0:
4     print("Tallet er ikke et oddetall!")
5 else:
6     sum = 0
7     i = 1
8     while(i <= tall):
9         sum = sum + i
10        i = i + 2
11
12 print(sum)
```

```
1 Skriv inn et oddetall: 5
2 9
```

Fasit 2.10.6:

```
1 x0= 1
2 b = 1024
3 x = 0
4 x1 = x0
5
6 while x1 != b:
7     x += 1
8     x1*= 4
9
10 print(f"Svaret er {x}.")
```

```
1 Svaret er 5
```

Fasit 2.10.7:

a) Resultatet blir:

```
1 A
2 B
3 B
4 B
5 A
6 B
7 B
8 B
```

b) Resultatet blir:

```
1 51 52 53 61 62 63
```

c) Resultatet blir:

```
1 51 52 53
2 61 62 63
```

Fasit 2.10.8:

a) Resultatet blir:

```
1 0 1 2 3 4 5 6 7 8 9
```

b) 1 2 3 4 5 6 7 8 9

c) 1 0 2 4 6 8

d) 1 10 9 8 7 6 5 4 3 2

Fasit 2.10.9:

```
1 svar = ""
2
3 while svar != "nei":
4     svar = input("Hei! Vil du fortsette programmet? ")
5
6 print("Du tastet nei, programmet avsluttes")
```

Fasit 2.10.10:

```
1 a1 = 3
2 k = 2
3 n=5
4 Sum=0
5 for i in range(n):
6     an=a1
7     an = a1 * k
8     Sum+=an
9
10 print(f'Summen til {n} ledd i en geometrisk rekke med a1 {a1} og kvotient {k}
      er {Sum} ')
```

Fasit 2.10.11:

Programkoden lister opp for hvert tall fra og med til og med 9, tallet fra (fra og med) 1 til (til og med) 4.

Fasit 2.10.12:

```
1 a1 = 5
2 k = 3
3 n=7
4 Sum=0
5 for i in range(n):
6     an=a1
7     an = a1+k
8     Sum+=an
9
10 print(f'Summen til {n} ledd i en geometrisk rekke med a1 {a1} og kvotient {k}
      er {Sum} ')
```

Fasit 2.10.13:

a) Programkoden lister opp for hvert tall fra og med til 1 og med 2, tallet fra (fra og med) 2 til (til og med) 6.

```
1 1 2
2 1 4
3 1 6
4 2 2
5 2 4
6 2 6
```

b) `range(5)` sender inn 0, 1, 2, 3 og 4 og resultatet blir for `i in range(5)`:

```
1 02438
```

c) `while i < 4` sender inn 1, 2, 3 og 4 og resultatet blir

```

1 1
2 2
3 3

```

while-delen erstattes med `for i in range(1,4):`

d)

```

1 1
2 2
3 3
4 4
5 i er ikke lenger mindre enn 5

```

e)

```

1 1
2 1 2
3 1 2 3
4 1 2 3 4

```

f)

```

1 o
2 o o
3 o o o
4 * * * *
5 * * *
6 * *
7 *

```

g)

```

1 n=4;
2 for i in range(n):
3     for j in range(i):
4         print ('* ', end=" ")
5     print(' ')
6
7 for i in range(n,0,-1):
8     for j in range(i):
9         print('o ', end=" ")
10    print(' ')

```

Fasit 2.10.14:

a) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(n+1,1,-1):
4     for j in range(1,k-i+3,1):
5         print(j,end=' ')
6     print()

```

b) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(1,n+1,1):
4     for j in range(k-i+1,0,-1):
5         print(j,end=' ')
6     print()

```

c) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(n+1,1,-1):
4     for j in range(k-i+2,0,-1):
5         print(j,end=' ')
6     print()

```

d) Programkode:

```
1 n = 4
2 k = 4
3 for i in range(n+2,0,-1):
4     for j in range(1,k-i+3,1):
5         print(j,end=' ')
6     print()
```

Fasit 2.11.1:

```
1 man får Skalarproduktet av a og b er 1.0 . Legg merke til  $-1 \cdot 0 + 3 \cdot 1 + 0 \cdot 5 = 1$ 
```

Fasit 2.11.2:

a)

```
1 import numpy as np
2
3 def A(r):
4     return np.pi*r**2
5
6 print(A(5))
```

b)

```
1 import numpy as np
2
3 r= float(input('Skriv inn radiusen til en sirkel: '))
4
5 def A(r):
6     return np.pi*r**2
7
8 print(A(r))
```

Fasit 2.11.3:

- a) Programkode 1 (0.25 ligger i konvergensintervall til s_1 . Programkode 1 og 2 (1.75 ligger i konvergensintervall for begge) Programkode 2 (2.25 ligger i konvergensintervall til s_2
- b) Ved å synke estimeringsnøyaktighet e i programkodene.

Fasit 2.11.4:

Programkode:

```
1 import numpy as np
2
3 def A(a,b,c):
4     s=1/2*(a+b+c)
5     A=np.sqrt(s*(s-a)*(s-b)*(s-c))
6     return A
7
8 a = 3
9 b = 4
10 c = 5
11
12 Areal=A(a,b,c)
13 print(f'Arealet til en trekant med sidelengdene {a}, {b} og {c} er {Areal}')
14 print(Areal)
```

Fasit 2.11.5:

Programkode:

```

1
2 def KE(m,v):
3     """
4     Denne funksjonen bestemmer kinetisk enegi for en partikkel gitt:
5     Input:
6         masse (kg)
7         hastighet (m/s)
8
9     Output:
10        Partikkelens kinetisk energi (J)
11        """
12        KE = 0.5 * m * v** 2
13        return KE
14
15 m= 0.0001
16 v=8
17 E=KE(m, v)
18 print(f'Kinetisk energi for en partikkel med massen {m} og hastighet {v} m/s er
        {E} J')

```

Fasit 2.11.6:

```

1 5
2 15

```

Fasit 2.12.1:

- a)
- `plt.plot(x,f,'g--')` tegner grafen til funksjonen og tar inn x -verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'` skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med grønnfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
 - Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
 - `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y -aksen.
 - `plt.grid()` legger på et gitter bak grafen.
 - `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
 - `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best')`: plasserer tekstboksen der det passer best.
 - `plt.show` tegner diagrammet i konsoll-vinduet.
 - `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.
- b)

Fasit 2.12.2:

```

a) 1 import matplotlib.pyplot as plt
   2 import numpy as np
   3
   4 xstart = -2
   5 xslutt = 2
   6 steglengde = 0.01
   7
   8 def f(x):
   9     return x**3 + 6*x - 3
  10
  11 x = xstart
  12 y = f(x)
  13

```

```

14 while x <= xslutt:
15     x += steglengde
16     y = f(x)
17     print(f'{x:.3f} {y:.3f}')
18
19 x = np.arange(xstart, xslutt, steglengde)
20 y = f(x)
21
22 plt.plot(x,y)

```

b) Programkode:

```

1 xstart = float(input('Skriv inn nedre grense for x: '))
2 xslutt = float(input('Skriv inn øvre grense for x: '))
3 steglengde = float(input('Skriv inn steglengden: '))
4
5 def f(x):
6     return x**3 + 6*x - 3
7
8 x = xstart
9 y = f(x)
10
11 while x <= xslutt:
12     x += steglengde
13     y = f(x)
14     print(f'{x:.3f} {y:.3f}')

```

Fasit 2.12.3:

Programkode:

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Her skal vi definere en funksjon, og deretter plotte den
6
7 # Deler inn x-aksen i 100 deler (101 punkt)
8 x = np.linspace(-2*np.pi, 2*np.pi, num=101)
9
10 def f(x):
11     f = np.sin(2*x - np.pi)
12     return f
13
14 plt.plot(x, f(x), color='green')
15
16 def g(x):
17     g = np.cos(2*x + np.pi)
18     return g
19
20 plt.plot(x, g(x), color='blue')
21
22 def h(x):
23     h = np.cos(2*x + np.pi/2)
24     return h
25
26 plt.plot(x, h(x), color='red')
27
28 plt.show()

```

f og h representerer samme funksjon.

```

1 import numpy as np

```

```

2 import matplotlib.pyplot as plt
3
4 c= 60
5 a=120
6 T= 365
7 k =2
8 d= 182.5
9
10
11 # Deler inn x-aksen i 1000 deler (1001 punkt)
12 x = np.linspace(0,T, num=1001)
13
14 def f(x):
15     f= c+a*np.sin((2*np.pi*k/T)*x+ d)
16     return f
17
18 plt.plot(x, f(x), color='blue')
19 plt.grid()
20 plt.xlabel('x ')
21 plt.ylabel('y')
22
23 def g(x):
24     g= c+a*np.cos((2*np.pi*k/T)*x+d)
25     return g
26
27 plt.plot(x, g(x), color='red')
28
29 plt.legend(loc='best')
30 plt.legend(['f(x)', 'g(x)'])
31
32 plt.show()

```

Fasit 2.13.1:

- a) [1, 2, 3, 4, 5, 2, 4, 6, 8, 10]
- b) A=[1,2,3,4,5]
- c) Programkode:

```

1 liste = [1, 2, 3, 4, 5, 6]
2
3 produkt = 1
4
5 for t in liste:
6     produkt *= t
7
8 print(produkt)

```

Fasit 2.13.2:

```

1 def f(x):
2     return -.2*x**2+1
3
4 xstart = -1
5 xslutt = 2
6 xsteg = .25
7
8 xverdier = []
9 yverdier = []
10
11 x = xstart
12 y = f(x)

```

```

13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)

```

Fasit 2.13.3:

```

1 def f(x):
2     return -x**3+x+2
3
4 xstart = -2
5 xslutt = 3
6 xsteg = .25
7
8 xverdier = []
9 yverdier = []
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)

```

Bibliografi

- [1] Amir M. Hashemi and Arne Hole (2025). Matematikk for ingeniørutdanning, 1. utgave. Fagnbokforlaget 2025, ISBN 9788245052114.
- [2] BBC Bitesize (2019). KS3 Computer Science. Introduction to computational thinking. Hentet fra <https://www.bbc.co.uk/bitesize/topics/z7tp34j>
- [3] Biologiske fagtermer fra A til Å. Institutt for biovitenskap ved matematisk-naturvitenskapelig fakultet ved UiO. Hentet fra <https://www.mn.uio.no/ibv/tjenester/kunnskap/plantefys/leksikon/>.
- [4] Collins Online Dictionary. Hentet fra <https://www.collinsdictionary.com/dictionary/english>
- [5] Martinez, S. L & Stager, G. S. (2019). Invent to learn (2. Utgave). Torrance, California, USA: Constructing Modern Knowledge Press
- [6] Jakobsen, Renate E. (2019). Programmering i matematikk –muligheter og utfordringer (Masteroppgave, UiA). Henter fra <https://uia.brage.unit.no/uia-xmlui/bitstream/handle/11250/2646381/Renate20Elise20Jakobsen.pdf>
- [7] Kunnskapsdepartementet (2018). Kjerneelementer i fag. Hentet fra <https://www.regjeringen.no/no/aktuelt/forny-er-innholdet-i-skolen/id2606028/>
- [8] Papert, Seymour. Mindstorms: Children, computers, and powerful ideas. Basic Books, Inc., 1980. Hentet fra <http://worrydream.com/refs/Papert20-20Mindstorms201st20ed.pdf>
- [9] Programmering som kompetanse for det 21. århundre. Udir, Notat nr. 2 - Programmering i skolen. November 2016. <https://www.udir.no/kvalitet-og-kompetanse/profesjonsfaglig-digital-kompetanse/notat-om-programmering-i-skolen/>
- [10] Wing, Jeannette (april 2008). «Computational thinking and thinking about computing». 2008 IEEE International Symposium on Parallel and Distributed.

Dette heftet vil fremme læreglede for å lære seg algoritmisk tenkning, modellering og problemløsning, og vil bruke programmering til å vise hvordan man kan løse oppgaver og problemer fra realfag og andre fag i ungdomstrinnet og videregående skole trinnet.

Problem: Skriv en programkode som regner ut volumet til en kule med radius r , og skisserer grafen til $V(r)$, for $0 \leq r \leq 5$.

Matematisk modell: $V(r) = \frac{4}{3}\pi r^3$

Programmering:

```
1 import matplotlib.pyplot as plt
2 from math import pi
3
4
5 # Leser opp radiusen fra skjermen
6
7 radius = float(input('Radius til kule:'))
8
9 # Funksjon som regner ut volumet til en kule med radius r
10
11 def volum(radius):
12
13     return (4/3) * pi * radius **3
14
15 V=volum(radius)
16
17 # Leser opp radiusen fra skjermen
18
19 print(f"Volumet til en kule med radius {radius} er: ", V)
20 r_verdier = np.arange(0,5.2,.2)
21 print(r_verdier)
22
23
24 #for r in r_verdier:
25 # formatering   print(f"{x:.2f}\t{f(x):.2f}")
26
27 r=r_verdier
28 V=volum(r)
29
30
31 #print(f'{r:.2f}\t,{V:.2f}')
32
33 plt.plot(r,V,'.-', linewidth=3, label='V(r)')
34 plt.grid()
35 plt.xlabel('r')
36 plt.ylabel('V')
37 plt.legend(loc='best')
38 plt.show()
```

Struktur og progresjon i oppgavene skal forberede deltagere til å kunne løse utfordrende problemstillinger og tilegne seg utholdenhet, kreativitet, selvtillit, og vil bidra til matematisk tenkning.

Programmeringsdic

Problem: vi skal se på filen 'tekopp-temperatur.csv'. Hver linje i filen er en temperatur-måling i en te-kopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 Grad celsius. Her bruker vi Newtons avkjølingslov for å studere temperaturendringen i vannkoppen.

Matematisk modell:

$$\frac{dT}{dt} = k(T - T_o)$$

der $T(t)$ ((målt i Grad celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

Programmering:

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 df = pd.read_csv('http://home.hib.no/ansatte/ahas/MP/tekopp-temp.csv', sep=
    ';', decimal='.', comment='#')
5
6 tid = df['Tid'].tolist()
7 temperatur = df['Temperatur'].tolist()
8 tstart = 0
9 Tstart = temperatur[0]
10 tslutt = 3700
11 deltat = 0.5
12 To = 22 # Rom temperatur
13 k = .0015 # proporsjonalitets koeff.
14 tverdier = []
15 Tverdier = []
16 t = tstart
17 T = Tstart
18 while t <= tslutt:
19     tverdier.append(t)
20     Tverdier.append(T)
21     t = t + deltat
22     deltaT = -deltat * k * (T - To)
23     T = T + deltaT
24
25 plt.plot(tverdier, Tverdier, linewidth=1, color='blue', label='T(t)')
26 plt.plot(tid, temperatur, 'x', color='red')
27 plt.grid()
28 plt.xlabel('t (sek)')
29 plt.ylabel('T (Grad celsius)')
30 plt.legend(loc='best')
31 plt.show()
```

